XIMOD Translator

User Manual

Cross-platform mod translation tool for Bethesda games

Skyrim · Fallout · Starfield · Oblivion · Morrowind

Version 1.5 — stage 1, batch 5

Cross-platform: Windows · Linux · macOS



Contents

1. Introduction 5
1.1 Who this tool is for 5
1.2 Supported games 5
1.3 Supported formats 5
2. Installation and startup 5
2.1 Contents of the application folder 5
2.2 Starting the application 6
2.3 First launch 6
2.4 Detecting games 6
2.5 Where your settings are stored: portable or per user 7
2.6 Interface languages 7
3. Key concepts 7
4. Interface overview 8
4.1 The menu bar 8
4.2 The toolbar 8
4.3 The filter bar 9
4.4 Tabs 9
4.5 The status bar 9
4.6 Window behavior 9
5. Quick start 9
6. The toolbar: Game, Source, Target, Output 10
6.1 Output modes 10
6.2 Changing the Game 11
6.3 The Source: the languages the file provides 11
6.4 The Target: by country 11
6.5 The Output: requested at export 11
6.6 Undo and redo 12
7. Opening files 12
7.1 Open, drag and drop, recent files 12
7.2 Localized plugins and STRINGS 12
7.3 Auto-translation: the dictionary stack 12
7.4 Several documents 13
7.5 Recovery after an abnormal shutdown 13
7.6 The mod dictionary: the working memory 13
7.7 Interface files and text formats 14
8. The translation grid 15
8.1 The columns 15
8.2 Entering a translation 16
8.3 Book preview 17
8.4 “Unique lines” view 17
8.5 The edit window 17

8.6 Find and replace 19
8.7 Quality checks 20
8.8 Dialogues 21
8.9 Voices 22
8.10 Translating online 23
8.11 Reviewing a translation with someone else 24
9. Statuses and their colors 25
9.1 The provenance of translations 26
10. Filters and counters 27
11. Saving and exporting 28
11.1 Save as… 28
11.2 Export translation… 28
11.3 Apply the dictionaries… 29
12. The Dictionaries menu 29
12.1 Before you start 30
12.2 Menu overview 30
12.3 Converting between XIMOD and external tools 31
12.4 Building a dictionary 32
12.5 Exporting a game dictionary to STRINGS 33
12.6 Managing games 33
12.7 Worked examples 33
12.8 Managing dictionaries: the tree 34
12.9 Text-only databases 34
12.10 My personal dictionary and the dictionary editor 34
12.11 Mod update 35
13. The dictionary viewer 35
14. The spell checker 36
14.1 Installing a dictionary 36
14.2 Opening and using the spell checker 36
14.3 The personal dictionary 36
15. The explorers (Tools menu) 36
15.1 Tools ▸ Data ▸ Translatable fields per game… 36
15.2 Tools ▸ Data ▸ Text formats… 37
15.3 Tools ▸ Translate a FOMOD… 37
15.4 Tools ▸ Package a translation… 38
15.5 Tools ▸ Translate the interface… 39
16. Settings 40
16.1 General tab 41
16.2 Recent files tab 41
16.3 Advanced tab 41
16.4 Online tab 42
16.5 Keyboard navigation 42
17. The configuration file 43

18. Localization of Bethesda games.....44
18.1 Where the application finds the STRINGS.....45
18.2 Knowing the language of a plugin.....45
18.3 Adding a language that Bethesda does not offer to a game.....46
19. Frequently asked questions.....47
20. Glossary.....47
Appendix — Summary of formats.....48

1. Introduction

XIMOD Translator is a desktop tool for translating mods (extensions) of Bethesda games. It reads the plugins and the translation databases produced by the community's reference tools, presents each text string in an editable grid, then writes back a translated version ready to be installed in the game.

Designed to be cross-platform (Windows, Linux, macOS) and free of proprietary dependencies, it brings the mod translator's complete workflow together in a single application: opening, dictionary-assisted translation, spell checking, status management and export in the formats the game engines expect. It is the companion of XIMOD Architect (FOMOD installer creation). The interface is available in French and English.

1.1 Who this tool is for

Anyone who wants to translate a mod, whether a beginner or experienced. No programming knowledge is required: you open a file, type the translations in a grid, and the tool takes care of the rest. Experienced translators will find the concepts of xTranslator and ESP-ESM Translator (statuses, databases, STRINGS, FormID).

1.2 Supported games

XIMOD Translator knows the following Bethesda games:

- Morrowind, Oblivion and **Oblivion Remastered**;
- Fallout 3, Fallout New Vegas, Fallout 4, Fallout 76;
- Skyrim (Legendary Edition) and Skyrim Special Edition / Anniversary Edition;
- Starfield.

The *Game* you select determines the character set (encoding) and the dictionaries loaded. The chapter “Localization of Bethesda games” explains how each generation stores its texts.

1.3 Supported formats

As input, XIMOD Translator reads:

- translation databases `.sst` (xTranslator), `.eet` (ESP-ESM Translator), `.dsd` (Dynamic String Distributor), `.json` (pivot or DSD) and `.xml` (EET, xTranslator and native dialects);
- plugins `.esp` / `.esm` / `.esl` (extraction of the translatable strings);
- string tables `.strings` / `.dlstrings` / `.ilstrings`;
- archives `.bsa` / `.ba2` (through the archive explorer).

As output: `.sst`, `.eet`, `.dsd`, `.json`, `.csv`, XML, as well as translated plugins and STRINGS files. The complete table is in the appendix.

2. Installation and startup

2.1 Contents of the application folder

The application comes as a self-contained folder containing:

- the executable `ximod-translator` (or `ximod-translator.exe` on Windows);
- the `assets/` folder with the shipped data: interface languages (`assets/locales`), game sheets and translatable field tables (`assets/data/games`, `assets/data/fields`), fonts, images and the splash screen (`assets/images/splash.png`); the dictionaries (`assets/bases/jeux` and `assets/bases/mods`) are stored there in portable mode;
- the `Manuals/` folder: this manual in French (`XIMOD_Translator_Manuel_FRA.docx`) and in English (`XIMOD_Translator_Manual_ENG.docx`), which *F1* opens in the interface language (§ 2.6);
- optionally the `Config.ini` file: its presence next to the executable makes the application run in **portable mode** (§ 2.5).

> **Note —** No system installation is needed: copy the folder wherever you want and run the executable. On Windows, an installer is also offered (installation in Program Files, shortcuts, optional association of `.ximdb` files); it installs no game dictionary, which you build from the Dictionaries menu or install separately. The application degrades gracefully if an optional file is missing (built-in fonts by default, no splash, default icon).

2.2 Starting the application

- **Windows**: double-click `ximod-translator.exe`.
- **Linux**: make the file executable, then run it (`./ximod-translator`). The usual egui/OpenGL graphics libraries (X11/Wayland) must be present.
- **macOS**: open the `.dmg` image and drag “XIMOD Translator” to Applications. Since the application is not signed, you launch it the first time by right-clicking ▸ *Open*, then confirming.
- **Open with**: one or more files passed as arguments (or dropped on the executable) open at startup.

2.3 First launch

1. A splash screen is displayed while the application detects the installed games and prepares the dictionaries. Its duration can be set in the Settings (it can be set to zero).
2. Open *Options* ▸ *Settings…* and, if needed, fill in the folders of the external tools and of the databases.
3. Choose the *Game* in the toolbar (at launch, the four lists are empty), then the country of the target language with the *Target* button and the *Output* mode: with no document open, these choices apply to the next file.
4. Open a file to translate (*File* ▸ *Open…* or drag and drop) — these commands become available only once a game is chosen. The *Source* language fills itself in from the file.

2.4 Detecting games

The *Dictionaries* ▸ *Detect installed games…* menu automatically searches for your games on all platforms: Steam (Windows, Linux — native, Flatpak and Snap installations and secondary libraries, Proton games included — and macOS), GOG (Windows registry, GOG Galaxy database on Windows and macOS), Epic Games (Windows, macOS), Heroic Games Launcher (GOG, Epic and Amazon, on Linux, macOS and Windows), Lutris (Linux), Xbox Game Pass (Windows), then a scan of folders. Each location is kept only if the game's master file is there (`Skyrim.esm`, `Fallout4.esm`…); a Windows game installed through Wine or Proton is validated on its real folder. *Add a game…* lets you designate one manually by pointing to its installation folder; a game added this way is locked and never overwritten by detection. The registered games feed the *Game* selector.

2.5 Where your settings are stored: portable or per user

The application writes its configuration, your personal dictionaries, its backup files, the data files you modify and its log to a **configuration folder** that depends on the mode:

Mode	Configuration folder
Portable	The executable's folder, as soon as a Config.ini is in it. This is the case for existing installations: nothing changes for them.
Per user	Windows: %APPDATA%\XIMOD Translator; Linux: ~/.config/ximod-translator; macOS: ~/Library/Application Support/XIMOD Translator. This is the mode of an installation in Program Files or in Applications.

The folder contains `Config.ini` and the subfolders `data/` (modified data files, for example the field tables, and the default dictionaries in per-user mode), `autosave/` (backup files), `dict/` (personal dictionaries) and `logs/` (startup log). You switch from one mode to the other in *Settings* ▸ *Advanced* (§ 16.3): the configuration is copied and the change takes effect at the next startup. If the folder is not writable, the application starts anyway, with the configuration read-only, and says so.

2.6 Interface languages

The interface exists in thirty-two languages, those of XIMOD Architect except Arabic: Bulgarian, Chinese, Croatian, Czech, Danish, Dutch, English, Estonian, Finnish, French, German, Greek, Hungarian, Irish, Italian, Japanese, Korean, Latvian, Lithuanian, Maltese, Norwegian (Bokmål, and “nor”), Polish, Portuguese, Romanian, Romansh, Russian, Slovak, Slovenian, Spanish, Swedish and Turkish. Each language is a file `assets/locales/<code>/main.ftl` (ISO 639-3 code) headed by a block giving the code, the name of the language, the font and the author. It is chosen in *Settings* ▸ *General* ▸ *Interface language* (§ 16.1); the change applies at once, without restarting, including to the generated comments of the grid. Arabic is already translated and its file is shipped, but it is offered neither in this list nor as displayed language of the translation editor until the interface can be drawn right to left; a Config.ini that still names it opens the interface in English.

The terms shared by both applications (menus, buttons, settings, countries and languages) reuse the translation of Architect, reviewed by its community. The others were machine-translated: the header says so (`# @author = … (traduction automatique, à relire)`) and the interface translation editor (§

15.5) lets anyone correct them. A message missing from a language is shown in English, the reference language; French is a complete translation of it.

Each language is displayed in its font (`Languages.json`, `font` field): Noto Sans for the Latin, Cyrillic and Greek scripts, Noto Sans JP, KR and SC for Japanese, Korean and Chinese, all loaded at start-up from `assets/fonts/`. Every text of the application is in these files, none is written in the program: menus and buttons, but also the error messages of the file readers and archives, size units (B, KB, MB), the date format, the names of the modifier keys (Ctrl, Shift, Alt), the punctuation that joins two messages (":" with no space in English, with a space in French). Only technical identifiers (record signatures, file names, addresses) and the diagnostic log, written in English, stay outside the language files. The manual exists in French and English: *Help* ▸ *Manual* (*F1*) opens the one in the interface language, English for the other languages.

3. Key concepts

A few notions are enough to understand how the whole tool works.

Term	What to remember
Plugin	The mod's file (.esp/.esm/.esl). It contains the game objects and, sometimes, their texts.
Translation database	A file that associates original texts with their translation (.sst, .eet, .dsd, .xml…). It serves as a reusable memory.
Dictionary (.ximdb)	XIMOD's own compact database, stored per game. It automatically pre-fills the translations already known.
STRINGS	The external text tables (.STRINGS/.DLSTRINGS/.ILSTRINGS) of recent games; one per language.
Localized / non-localized	A "localized" plugin stores its text in the STRINGS and keeps only identifiers; a non-localized plugin contains the text in plain form.
Status	The state of a line (to translate, validated, pending…), shown by a color and a glyph.
Output mode	The format produced on export (full plugin, patch, STRINGS, DSD…).
FormID / EDID	Technical identifiers of a record, used to match a translation precisely to its string.

4. Interface overview

From top to bottom, the main window is organized into several areas.

4.1 The menu bar

Five menus: *File* (open, save, load a review, export, recent files, exit), *Options* (settings, colors, special characters, optional columns, spell checker, databases and applying dictionaries), *Dictionaries*

(conversions, creation, viewing, tree and editing of dictionaries), *Tools* (find and replace, checks, dialogues, online translation, explorers and, under *Data*, the data file editors) and *Help* (manual, about).

Menu shortcuts. The same as in XIMOD Architect, shown to the right of the entries (⌘ on macOS): *Ctrl+O* Open…, *Ctrl+S* Save as…, *Ctrl+Shift+E* Export translation…, *Ctrl+W* Close document, *Ctrl+Q* Exit, *Ctrl+,* Settings…, *Ctrl+H* Find and replace…, *F12* Quality checks…, *F1* User manual (the file in the `Manuals` folder in the interface language). They do nothing while a dialog is open. Closing a document or exiting with unsaved changes first asks: Save as… (Enter), Close without saving, Cancel (Esc).

4.2 The toolbar

Four lists: *Game*, *Source* (original language), *Target* (translation language) and *Output* (export format), followed by the ↺ *Undo* and ↻ *Redo* buttons. They show and change the settings of the **active document**; with no document, the settings of the next file. The label on the right says which ("Settings of MonMod.esp" / "Settings of the next file"). When the application starts, and as soon as the last tab is closed, the four lists are empty. *Source* offers only the languages the open file provides; *Target* is a button that opens the flags window, and the list to its right contains only the official languages of the chosen country; *Output* can stay empty until export, which asks for it. Only the *Game* is required to open files. Details in chapter 6.

4.3 The filter bar

The *Filter* field searches for a text in the target chosen in the *In* list: *All* (source, translation, EDID and record signature), *Source*, *Translation*, *EDID or FormID* — a text of six to eight hexadecimal digits, with or without `0x`, is compared with the FormID, with or without the master index ("0001A3F2" and "01A3F2" find the same line); any other text is searched for in the EDID —, *Signature* ("WEAP:FULL", "WEAP" or ":FULL"), or *Comment* — the text of the Comment column (§ 8.1), author's comment and generated text combined, which allows "voice ✗", "female", "stage 20" or "master: Skyrim.esm"; this target triggers the context pass (§ 8.8) if it has not run yet, and works whether or not the column is shown. The *.** box reads the field as a regular expression (Rust syntax, case-insensitive; `(?-i)` at the start to match case): "(dragon|draugr)" finds both. The filter applies a quarter of a second after the last keystroke, or immediately on Enter; an invalid expression does not empty the grid: the view keeps the last valid filter and the field turns red, with the error in a tooltip. *Ctrl+F* gives focus to the field and selects its content, *Esc* in the field clears it, as does the red cross; *Reset* sets the target back to All and turns off the expression. The target and the option last for the session. After the statuses, the *[!] Anomalies* button keeps only the lines flagged by the last quality check (§ 8.7, F12); Reset releases it. On a 50,000-line file, a filter applies in a few milliseconds, and the view is recomputed only when the filter or the document changes.

Located below the toolbar, it appears as soon as a file is open and restricts the lines displayed (chapter 10).

4.4 Tabs

Each open file gets its own tab; click it to switch, or click the cross to close it. Next to the name, a discreet badge recalls its settings ("Skyrim SE · ENG→FRA · STRINGS"), and a ● dot precedes the name as long as

the content differs from what was saved. Right-clicking the tab gives access to the *Use my personal dictionary* box (§ 7.3). Known DLCs show their plain name (the real file name remains in a tooltip).

4.5 The status bar

To the right of the counters, "● Modified" signals an unsaved document (the "●" also appears in the window title and on the tab). Important outcomes — save, export, errors — are also shown for a few seconds at the top right (transient notifications: 4 s for a success, 7 s for a warning; an error stays until you close it); the *Transient notifications* box in the Settings turns them off.

At the bottom: on the left, the last message (import, export, auto-translation…), cut off with an ellipsis if it is too long for the window (the full text appears in a tooltip); on the right, always visible, the counters — lines *shown* (out of the total), *unique*, *to translate* and *translated*.

4.6 Window behavior

Secondary windows (Settings, dictionary viewer, explorers, spell checker…) are **independent**: they open next to the main window, can be moved and resized freely, and their size and position are remembered in `Config.ini`. You can operate the dialogs from the keyboard (Tab, Esc, Enter, arrow keys).

5. Quick start

To translate your first mod, from start to finish:

1. Choose the *Game*, the *Source* language and the *Target* language in the toolbar.
2. Open the mod's file (*File* ▸ *Open…* or drag and drop). The game is recognized from the file; the lines already known to this mod's dictionary, to your personal dictionary and to the game database are pre-filled automatically.
3. In the grid, click in the *Translation* column and type the text, line by line (*Ctrl+Z* undoes). Use the filters to see only the *To translate* lines, then set the reviewed lines to the *Validated* status: only these are exported by default.
4. Check the spelling with *Options* ▸ *Spell checker…*.
5. Choose the *Output mode* suited to the game, then *File* ▸ *Export translation…*.
6. Install the result in the game (see chapter 11 and the MO2/Vortex rules specific to each game family).

6. The toolbar: Game, Source, Target, Output

6.1 Output modes

The *Output* selector chooses what *Export translation…* produces:

Mode	What it produces
Full plugin	A standalone, fully translated .esp/.esm plugin; for a localized plugin, the strings from the STRINGS are reinjected into the plugin (delocalization).

Mode	What it produces
Override patch	A patch plugin that contains only the translated records and loads after the original (it depends on it).
STRINGS	The translated .STRINGS/.DLSTRINGS/.ILSTRINGS files, to be installed next to the localized plugin. Also works from a STRINGS document on its own.
Hybrid	Like STRINGS, but from a loaded plugin (record/field context). Requires an open plugin.
Text file	The translated interface file (MCM, INI, JSON…, § 7.7), written in the target language next to the source file: the only mode offered for text documents, absent from the list for plugins.
DSD (SKSE)	A Dynamic String Distributor file (JSON, Skyrim SE/AE) injected at launch through SKSE, without redistributing the original mod.

> **Note —** The *DSD (SKSE)* mode applies only to Skyrim SE/AE; a warning is shown if the active game does not support it.

Which mode for publishing? To distribute the translation of a mod you did not write, choose *Override patch* (or *STRINGS* for a localized plugin, or *DSD* on Skyrim SE): the original mod stays intact, your plugin contains only the translated records, copied as overrides, and it keeps applying when the author updates the mod. This is what authors expect from a translation, and it is the norm on Nexus. *Full plugin* rewrites the original: keep it for your own mods, or for games without STRINGS when the author accepts it. The confirmation shown at export reminds you of this when this mode is selected.

Each open document carries its own Game, Source, Target and Output settings. On opening, the game is the one recognized in the file (or the one in the *Game* list if recognition is impossible), the *Source* is deduced from the file (§ 6.3), and *Target* and *Output* take the values of the lists. Changing a list while a document is active changes **only that document**; with no document, it sets the setting of the next file. You can thus keep a Skyrim SE mod and a Fallout 4 mod open side by side, each with its own game and export mode.

6.2 Changing the Game

Changing the *Game* of a document **reloads it as that game**: the file is reread with the new game's rules (translatable fields, encoding), the translations already entered are kept by matching (FormID, then EDID, then context, then text), and the new game's dictionaries complete what is still empty. If the document has been modified, a confirmation comes first: *"Reload MonMod.esp as Fallout 3? Translations will be kept by matching."* The other documents are not affected; nothing is closed. This is useful for ambiguous cases (Fallout 3 / New Vegas, Oblivion / Oblivion Remastered) that automatic recognition cannot settle.

> **Note —** This reload clears the document's undo history, since its lines have been re-extracted.

6.3 The Source: the languages the file provides

The *Source* list offers only the languages that the open file actually provides; with no document, it is empty and inactive. For a **localized plugin**, these are the languages of the STRINGS files found next to the plugin or in its archive (a plugin can offer several). For a STRINGS file on its own or an xTranslator XML, it is the

language written in its name or in its `<Source>` tag. For a **non-localized plugin** or a binary database (SST, EET), which do not state their language, the application **detects it in the text** using the Lingua library (75 languages): the list receives the plausible language or languages, with the most likely one selected. The status bar reports the language chosen when it differs from the one that was displayed.

Changing the *Source* of a document re-resolves its STRINGS in the new language and completes its lines that are still empty. Detection is an estimate: on a mod made only of short names, check the proposed choice.

6.4 The Target: by country

Target is a button: it opens the flags window (the same one as in the Settings). Clicking a flag closes the window and places in the list to the right of the button the **official languages of the country**, under their endonyms (“Deutsch”, “Português”…). A country with a single language is selected immediately; for a country with several official languages (Belgium, Switzerland, Canada…), the list offers them and the status bar mentions them: choose one. Every language thus has its place, whether or not Bethesda publishes it (§ 18.3).

Changing the *Target* language of a document rebuilds (or retrieves from memory) the dictionary index of the game / target pair and completes its lines that are still empty **without ever touching the translations already entered**. As long as no target is chosen, the dictionaries do not apply (the status bar says so), and the commands of the *Dictionaries* menu that name a language pair ask for the target.

6.5 The Output: requested at export

The *Output* list can stay empty. *File ▸ Export translation…* is what decides: if the list is empty, an alert asks you to choose the output; if it is set, a confirmation recalls the selected output (“The selected output is ‘STRINGS’. Is it the right one?”) — *Yes* continues to the window of exported lines (§ 11.2), *No* returns to the document. Changing the *Output* changes only the document's export mode; the DSD warning follows the document's game.

6.6 Undo and redo

The ↺ and ↻ buttons of the toolbar, the shortcuts *Ctrl+Z* / *Ctrl+Y* (or *Ctrl+Shift+Z*; ⌘ on macOS) and the commands of the grid's context menu undo and redo changes to the content: typing in a cell (one operation per cell, from the start to the end of the input), status change, application of a database or dictionary, automatic re-translation, propagation to duplicates, spelling correction, resumption of an automatic save. Each document has its own history; the button's tooltip names the operation and the number of lines (“Undo: Application of the base game (3,214 lines) — Ctrl+Z”). After an undo, the grid scrolls to the first affected line, and the status bar indicates how many lines are outside the current filter. The tab's “modified” dot disappears when you return exactly to the saved state. The number of operations kept (200 by default) is set in *Settings ▸ Advanced*.

7. Opening files

7.1 Open, drag and drop, recent files

- Menu *File* ▸ *Open…*: translation databases, plugins, and text files (MCM, INI, JSON…, § 7.7).
- **Drag and drop**: drop one or more files on the window. A `.bsa` / `.ba2` opens the archive explorer; everything else is loaded into the translator.
- Menu *File* ▸ *Recent*: reopens a recently used file (the number is adjustable in the Settings).

> **Note —** As long as no game is chosen in the *Game* list of the toolbar, *Open…*, *Save as…* and *Recent* are greyed out (a tooltip explains why), drag and drop shows a warning, and the *Dictionaries* menu offers only *View a dictionary…*. The game determines the dictionaries loaded and the destination folders. The other three lists never block opening. From a command line or a shortcut, `--game SkyrimSE MonMod.esp` presets the game and opens the file immediately.

7.2 Localized plugins and STRINGS

For a localized plugin (Skyrim SE, Fallout 4, Starfield…), the fields contain string identifiers rather than the text. On opening, the application automatically looks for the `.STRINGS` files of the *Source* language next to the plugin (or in a neighboring `.bsa`/`.ba2` archive) to display the original text; the languages found make up the *Source* list (§ 6.3). The encoding is detected automatically (UTF-8, otherwise Windows-1252).

7.3 Auto-translation: the dictionary stack

On opening, the lines already known are pre-filled from the document's active dictionaries, in the priority order of its tree (§ 12.8): the first dictionary that knows the line wins, each level filling only what the previous ones left empty. In each dictionary, an exact match (identical record, field, FormID, EDID and source text) gives the Validated status — or the remembered status for the mod dictionary —, then a match by text (same record and field, same text) gives By similarity. The initial order of the roles:

1. the **mod dictionary**, `mods/<game>/<mod name>_<SRC>-<DST>.ximdb` — your own working memory on this mod, written at each save and export (§ 7.6), or the translation of an earlier version of the same mod; a line it recognizes exactly gets back the status it had (Validated, Pending, Custom, Ignored…);
2. your **personal dictionary** (`dict/personal_<SRC>-<DST>.ximdb` in the configuration folder, § 12.10), only if the *Use my personal dictionary* box of the tab's context menu is checked (unchecked by default; the initial value can be set in `Config.ini`); this is your terminology: a pair that matches exactly by text gives the Custom status;
3. the **text-only** databases: text → text pairs with no record or field (§ 12.9), which recognize a line whose source text is identical, whatever its field, with the By similarity status;
4. the **game dictionaries** of the game / target pair, `jeux/<game>/*.ximdb` whose declared language is the target, in alphabetical order;
5. the **external dictionaries**: those of the other mods in `mods/<game>`, and the files or folders you have added to the tree;
6. the **backups** of the mod dictionary (`backups/`), unchecked at first.

The status bar names the dictionaries that filled something, in order, with their number of lines ("Dictionaries: MonMod_ENG-FRA (212), personal_ENG-FRA (3), Skyrim_ENG-FRA (1,004) — 1,219 lines pre-filled"). You therefore only type what remains. Each pre-filled line remembers its **origin**: the tooltip of its status cell says "By similarity — Skyrim_ENG-FRA.ximdb, same text", and the Record tab of the edit window repeats it; a line filled by the Approximate mode (§ 11.3) says "approximate (75 %)"; this provenance is saved with the line and covers every way a translation arrives (§ 9.1). Each file is read only once per session (and reread if it changes on disk), so switching from one tab to another rereads nothing. *Options ▸ Reload translation databases* rereads the dictionaries from disk and replays the stack on the empty lines of the active document; this application can be undone.

7.4 Several documents

You can open several files; each has its own tab. The grid, the filters and the export apply to the active document (the highlighted tab).

7.5 Recovery after an abnormal shutdown

Every five minutes (adjustable), each document modified since its last automatic save is written in the background to the backup folder (`autosave/` in the configuration folder), without touching the original file or the "modified" state; the status bar shows it discreetly ("Automatic save 10:41"). When a tab or the application is closed normally, these files are deleted. After a crash or a power cut, the next startup offers *Resume your work?* with the list of files, the date and the number of lines: *Resume* reopens each file with its settings and its saved translations (an undoable operation; the document is marked modified); *Ignore* keeps the backups for thirty days without offering them again. If the original file has changed in the meantime, what matches by identifier is recovered and the rest is reported.

7.6 The mod dictionary: the working memory

Each open mod has its dictionary, `mods/<game>/<mod name>_<SRC>-<DST>.ximdb` in the mod dictionaries folder. It is rewritten at each *Save as…* and at each successful *Export translation…* (option *Write the mod dictionary at each save and export* in Settings ▸ Advanced, on at first), and on request through *Dictionaries ▸ Save the mod dictionary*. It contains **all** the lines that are no longer "To translate" — translated, whatever their status, or ignored — with their status; lines never shown in game are not included. The status bar says so: "Mod dictionary written: 214 line(s) → MonMod_ENG-FRA.ximdb". The previous file goes into `backups/` according to the number of timestamped copies (§ 16.3).

When the mod — or a later version of it — is reopened, this dictionary is applied first (§ 7.3): a line recognized exactly (identical record, field, FormID, EDID and source text) gets back its translation **and its status**, including Pending with no text or Ignored; a line whose source text alone matches (another record) gets the translation with the By similarity status, as from any other database. You therefore find your work as you left it, with no working file to reopen. The dictionaries created through the Dictionaries menu (§ 12.4), intended for sharing, carry no status: they contain only what is validated and are applied as Validated.

Nothing is lost when a mod is updated. At each write, the lines of the previous file that the document no longer has (a removed line of dialogue, a renamed object) are not discarded: they move to the **old data**

section of the same dictionary, with the current date, without duplicates; a line that returns in a later version comes back out and gets its status back. This old data is used after the current lines: a line recognized exactly gets back its translation and its status, an identical text is taken up by similarity, with the origin saying “old data”; a line whose text alone has changed is never filled on opening — the Mod update window (§ 12.11) or the FormID only mode (§ 11.3) proposes it. *Keep the old data of the mod dictionaries* (Settings ▸ Advanced, in months, 0 = forever, `[Backup] OldDataMonths`) purges it by age at write time; the editor (§ 12.10) shows it and deletes it entry by entry.

7.7 Interface files and text formats

Almost every Skyrim mod with a configuration menu (SkyUI MCM) and many Fallout 4 mods keep their interface texts outside the plugin, in `Interface/Translations/<mod>_<language>.txt`: an identifier “$Key”, a tab, the text, in UTF-16LE with a byte order mark; the game itself has `translate_<language>.txt`. Other mods use INI or JSON files. The program opens these files as full documents — grid, statuses, dictionaries, undo, edit window, checks — and writes them back in the target language while respecting the file: encoding, line endings, comments and foreign lines.

Opening. *File ▸ Open…* offers “Text files (MCM, INI, JSON…)”; drag and drop and *Recent* work the same way. The format is recognized from the file name (format patterns, § 15.2); when several formats match, or none, the *Which format for …?* dialog lists the formats with, for each one, the number of lines recognized among the first hundred, and shows the first lines of the file — *Open* or Enter takes the selected format. A `.json` file is first read as a database (pivot, DSD) and becomes a text file only if it is not one.

The grid. The columns become *Format* (the format identifier in capitals: MCM, INI, JSON…), *Field* (always TXT) and *Key* (the key in the file, “$MonMod_Title”, with its instance number when it is repeated). A line not recognized by the format's expression — comment, empty line, JSON header — does not appear but is kept in place for writing. The tab badge shows the format and the encoding (“MCM · UTF-16 · ENG→FRA”); the status bar shows “MCM / translate file: 184 keys · 120 translated · target: MonMod_french.txt (existing, 180 keys taken)”. Keys are never translated; the “{0}”, “{}” and “%s” placeholders in the values count as tags (Aliases and numbers tab, quality checks, `tags` list in `assets/data/fields/Text.json`); there is no length limit and line breaks are allowed. The spell checker, the dictionaries, the word heuristic, instant translation and propagation apply as they do to a plugin.

Languages. An MCM file name carries its language: lowercase English names for Skyrim (`english`, `french`, `german`…), codes for Fallout 4 and Starfield (`en`, `fr`, `de`…, `ptbr`) — the table is the one used for STRINGS (§ 18). The *Source* list offers the languages of the sibling files present (MonMod_english.txt, MonMod_german.txt), and choosing another one opens that file in place of the document (if it has no unsaved changes); *Target* is chosen as usual. When the target file already exists, its values pre-fill the translations with the Validated status, origin “target file” (status tooltip, Record tab), before the dictionaries; the keys it has and the source does not are counted and ignored. A `translate_<language>.txt` file, or a format without a language in its name, takes the languages from the toolbar — or, failing that, the language detected in the text.

Writing. *File* ▸ *Save as…* proposes `<mod>_<target language>.txt` in the file's folder (the file itself when its name carries no language), and *Export translation…* — Output "Text file" — does the same after the exported-lines dialog (§ 11.2), where the *Untranslated lines: keep the source* checkbox (`[Export] UntranslatedCopy`, checked by default) decides what happens to lines without a translation: the source is written, or the line is omitted. The file is written in its original encoding (MCM: UTF-16LE with byte order mark) and with its original line endings, the translated values replaced in place and everything else copied as is: a comparison with the original differs only in the translated values. Save as… also keeps the database formats (.sst, .eet, .xml…) so you can keep a record of the work. The mod dictionary (§ 7.6) is written as for a plugin, under the file's name (`MonMod_english_ENG-FRA.ximdb`), and joins the plugin's stack through the External role — and conversely: the plugin's dictionary pre-fills the MCM keys whose text is known.

Link with the plugin. When you open a plugin, the program looks for `Interface/Translations/<plugin name>_<source language>.txt` in the plugin's folder (all the mod's interface files when none is in the source language); if it finds any, the status bar says so ("Interface file found: MonMod_english.txt — File ▸ Open the mod's interface files") and the *File* ▸ *Open the mod's interface files* entry opens it in a new tab, with the plugin's game and languages; for a plugin without an interface file, the entry is grayed out. Nothing opens automatically.

8. The translation grid

8.1 The columns

The grid shows one line per string, with six fixed columns, to which the *Options* ▸ *Columns* sub-menu adds optional columns, all hidden at first — the same menu opens with a right-click on the grid header; the choices are remembered in `[Columns]`:

Column	Content
Status	Colored dot + glyph showing the state of the line (see chapter 9).
GRUP	Signature of the record (e.g. MESG, BOOK, INFO).
Field	Translated subfield (e.g. FULL, DESC).
EDID	Editor ID of the record, when the format provides it.
Source	Original text (Source language), on several lines if needed.
Translation	Input area for the translation (Target language).
Review	Optional (Options ▸ Columns ▸ Review), shown automatically when a review is loaded. The text proposed by a reviewer, editable like the translation; in blue when it differs from the translation, in grey when it is the same (§ 8.11).
Provenance	Optional (Options ▸ Columns ▸ Provenance). Where the translation came from: "dict. game (exact)", "online (DeepL)", "typed", "identical"…; the full label in the tooltip (§ 9.1).

Column	Content
Comment	Optional (Options ▸ Columns ▸ Comment). The author's comment, when the file carries one (EET, XML), then the generated context of the line: speaker and gender, quest, topic, emotion, response rank for a reply; model of an object; journal entry of a quest; master file of an override.

The Comment column. *Options* ▸ *Columns* ▸ *Comment* (`[Columns] ShowComment`, unchecked by default; the former `[Editing] ShowComment` key is taken over) adds the last column, after Translation and Provenance. Its text is generated on display from the dialogue map (§ 8.8) and from the subrecords read by the context pass, cached per line as long as the document and its map do not change, and costs nothing for lines off screen: a 50,000-line file stays smooth. The text appears once the context pass has finished (the column starts it on first request). Its content depends on the line type, unknown elements being omitted: a **reply** (INFO) gives “Ralof (m) · MQ101 · MQ101RalofTopic01 · Anger 60 · response 1 / 2” — speaker and gender, quest, topic (or category / subtype), emotion and intensity, response rank —, then “voice ✓” or “voice ✗” once the voice map is built (§ 8.9); a **topic** (DIAL) “MQ101 · MQ101RalofTopic01 · 3 replies”; a **quest** “quest MQ101 · journal entry, stage 20” for a journal text (CNAM) or “quest MQ101 · objective 10” for an objective (NNAM); a message, perk or spell tied to a quest “quest MQ101”; any **object with a model** (weapon, armor, book, container, plant, light, door…) “model: Weapons\Iron\IronWarAxe.nif”; an **NPC** “male · NordRace · voice MaleNord”; and any **override** of a record from a master file ends with “master: Skyrim.esm”. The author's comment, carried by EET and XML files since batch 1, comes first, separated by “·”; it cannot be edited here and is kept as is when saving. A text longer than 200 characters is truncated, and the tooltip shows it in full; the column width follows the window width.

8.2 Entering a translation

Click anywhere in the *Translation* cell — even at the bottom of a tall line — and type the text; the line's status changes automatically. The *Source* and *Translation* columns wrap long texts, so the input column is never pushed off screen. The grid is virtualized: even a file of several thousand lines stays smooth.

The line menu. A right-click anywhere on a line — dot, identifiers, source or translation — opens a single menu; if the line is not in the selection, it becomes the selection, otherwise the menu acts on the whole selection and says so (“Status of the selection (3 lines)”, “Paste into the 3 selected lines”). Five groups: *Edit* — Edit… (Enter: opens the edit window, § 8.5), Copy the source, Copy the translation (Ctrl+C), Paste into the cell or into the selected lines (Ctrl+V), Translation = source (Ctrl+F8: copies the source into the translation without changing the status, skipping Ignored lines); *Status* — Validated, Pending, Custom, Ignored (F5 to F8) and Validate then next (Ctrl+↓); *Dictionaries* — Apply the dictionaries to the selection… (§ 11.3), Add to my personal dictionary (Ctrl+M, § 12.10) and Apply to every line with the same source… (Ctrl+F6, § 8.4); *Navigation* — Filter on this record (all lines with the same FormID) and Filter on this source (all lines with the same source text); these filters appear as chips in the filter bar, and clicking a chip removes it; *Line tools* — Book preview (§ 8.3); then Undo / Redo. An entry that does not apply is grayed out and its tooltip says why (empty clipboard, not a book); the remaining dictionary entries (batch 3) and history entries (stage 2) will appear only with their batch. Any menu action that modifies the document

can be undone in one step, whatever the number of lines. The menu shortcuts Ctrl+C / Ctrl+V / Ctrl+F8 / Enter act only when no cell is being edited: inside a cell, they keep their usual meaning.

Selection and statuses from the keyboard. Clicking a line selects it (bluish tint); Shift+click extends the selection, Ctrl+click adds or removes a line; clicking in the *Translation* cell also selects the line. The function keys give a status to the whole selection: *F5* Validated, *F6* Pending, *F7* Custom, *F8* Ignored. A line without a translation cannot become Validated or Custom: it is skipped and the status bar says so ("3 lines → Validated; 1 without translation skipped"). *Ctrl+*↓ validates the line (or the selection) then moves to the next line in the view, *Ctrl+*↑ to the previous one; if you were typing, the input cursor follows: type, Ctrl+↓, type… translates a file without touching the mouse. At the end of the view, the status bar says "Last line of the view.". Each key press is an undoable operation ("Undo: Status Validated (3 lines)"), and a line whose status has just changed stays displayed even if the status filter now excludes it, until the next change of filter or selection.

Length and line breaks. As long as the translation respects the limits of its field, the cell shows nothing. When a limit is exceeded, the cell shows the count in red at its right edge ("34 / 32": characters against a character limit, or "37 / 32 o" against a byte limit); a line break in a field declared single-line is flagged by *[!]* in orange; an engine tag or a number that differs between the source and the translation is flagged by ⚠ (red: missing or unknown tag or number; orange: same tags in a different order, a mere warning). The cell's tooltip gives the details (field, limit, bytes, tags involved); the Aliases and numbers tab of the edit window (§ 8.5) offers the repairs. The limits and whether line breaks are allowed come from the game's field table (§ 15.1); a field with no known limit is never flagged. Exceeding a limit prevents neither input nor validation: you stay in control.

8.3 Book preview

For book records, a 📖 *Book preview* button opens a window showing the formatted *Source* and *Translation* side by side, with the *Game fonts* option for a faithful rendering.

8.4 "Unique lines" view

The *Unique* filter shows only one line per distinct source. Translating this unique line **automatically propagates** the translation to all its hidden duplicates — handy when the same text recurs many times.

Explicit propagation. From the normal view, *Ctrl+F6* (line menu ▸ Apply to every line with the same source…, in the grid and in the edit window) on a translated line opens a dialog that lists the document's other lines with the same source text — excluding Ignored lines and lines never displayed — with record, field, EDID, current translation and status. All are checked, except Validated and Custom lines whose translation differs; Check all / Uncheck all help. *Apply to the N checked lines* (Enter) copies the translation **and the status** of the original line onto the checked lines, in one undoable operation ("Undo: Propagation to duplicates (19 lines)"); the status bar gives the count. When there is no duplicate, the status bar says so ("No other line has this source."). On a selection of several lines, each translated line forms a group, each source once. The *Propose the propagation to the lines of the same source after a validation* option (Settings ▸ General, `[Editing] ProposePropagation`, unchecked by default) opens the same dialog after F5 or Ctrl+↓ on a single line that has differing duplicates; Esc closes it without doing anything and does not keep you from moving on.

8.5 The edit window

For long texts, or to translate a file in one go without leaving the keyboard, a separate edit window accompanies the grid. *Enter* on the selected line, a double-click on a line (outside its translation cell) or *Edit…* in the line menu opens it on that line — or brings it to the front if it is already open; the cursor is placed at the end of the translation. There is only one, tied to the active document: switching document tabs switches the window's document, and *File* ▸ *Close* closes it along with the document.



Figure 8.5 — The edit window: line identity, source, translation with its counter, Record tab, navigation and statuses.

From top to bottom: the line identity (record and field, EDID, FormID, instance number, originating plugin for a DSD); the *Source*, read-only but selectable and copyable; the *Translation*, multiline, where your input is carried over to the grid live — the carry-over becomes an undoable operation (§ 6.6) when the line, the status or the focus changes, and *Ctrl+Z* in the window undoes the last one; below the area, the counter: “146 / 512 chars · 151 bytes · line break: allowed” when the field limit is known, “146 chars · 151 bytes” otherwise — orange from 90% of the limit, red beyond it, *[!]* on a forbidden line break; bytes are counted in the game's encoding (UTF-8 for Skyrim SE, Fallout 4, Fallout 76 and Starfield; Windows-1252 for the older games, where one character is one byte); the context tabs; finally *Previous* / *Next* and the four status buttons, the current status shown raised.

From the keyboard. *Ctrl+Enter* validates the line then moves to the next line in the filtered view: type, Ctrl+Enter, type…; *Alt+Right* / *Alt+Left* change line without touching the status; *F5* to *F8* set the status of the displayed line; *Esc* closes the window and keeps your input. The window and the grid stay in step: the displayed line is the selected line, the grid scrolls to show it, and clicking another line in the grid changes the window's line. At the end of the view, the window stays on the last line and says so. *Enter* alone inserts

a line break when the game's field table (§ 15.1) allows it, or as long as the table says nothing; in a field declared single-line, nothing is inserted and the window reports it.

The *Record* tab first shows, below the origin of the translation, the line's comment ("Comment: Ralof (m) · MQ101 · …", § 8.1 — the author's comment, then the generated text, the latter as soon as the dialogue map is built), then lists the other translatable fields of the same record (same FormID), with their status, source and translation; clicking a field name goes to it. The *Aliases and numbers* tab compares the engine tags and numbers of the source with those of the translation: "`<Alias=Victim>` missing from the translation", "`<Alias=Victime>` missing from the source", "Number 3 missing from the translation", "Tag order differs from the source" (a warning: word order changes from one language to another), or "consistent". *Restore aliases* replaces the i-th tag of the translation with the i-th tag of the source, adds the missing ones at the end of the text when the translation has fewer, and says so; *Restore numbers* does the same for numbers; each repair is an undoable operation. The recognized tags — `<Alias…>`, `<Global…>`, `<Token…>`, `<BaseName>`, `<Pronoun…>`, book formatting tags — are the `tags` key of the game's field file (§ 15.1), compared case-insensitively; an empty translation is never an anomaly. The *Dialogue* tab (§ 8.8), on an INFO reply, summarizes its context — speaker, gender and voice, quest, topic, response rank, emotion — then gives the player's question, the reply before and the reply after, the conditions in plain language, the voice and the author's note; *Previous* / *Next* (Ctrl+Alt+Left / Ctrl+Alt+Right) change the window's line by following the conversation, and *Dialogues…* opens the Dialogues window; on a topic or a quest, the tab gives its label and counters; on any other line, "This line does not belong to a dialogue.". The *Vocabulary* tab offers, a tenth of a second after you arrive on the line, the sources close to the line in the active dictionaries (§ 12.8), with their translation: a **word heuristic** compares the words of the two texts (ignoring case and punctuation; an engine tag counts as a "tag" word, a number as a "number" word) and keeps the sources whose number of differing words — missing, extra or replaced — does not exceed the *Word heuristic threshold* (Settings ▸ Advanced, 20% by default, from 5 to 50; `[Fuzzy] Threshold`). Each line shows the score (green at 90% and above, yellow from 70 to 89%, orange below), the source with its differing words underlined, the translation, the dictionary and the original field ("(old)" for old data, § 7.6), and a *Use* button that copies the translation into the line with the Pending status — to adapt, then validate — in one undoable operation. Eight suggestions at most, sorted by score, then by stack order; words that are too frequent in a dictionary (present in more than 5% of its lines) are not used to preselect candidates. A dictionary's word index is built in the background on first request ("Word index in progress (42 %)…"), a few seconds for a hundred thousand lines, and lasts for the session. Below the suggestions, the **Known fragments** line lists the parts of the source that the dictionaries know ("Satchel → Sacoche (Skyrim_ENG-FRA) · of Moon Sugar → de sucrelune (personal_ENG-FRA)"): this is what instant translation will produce. *Dialogue* will arrive with its batch. A right-click in the Translation area opens the Edit and Status groups of the line menu (§ 8.2). The window geometry is remembered (`[Window] EditX/EditY/EditW/EditH`, 760 × 520 by default, opened offset near the main window), as is the active tab (`[Editing] EditTab`).

Instant translation. You can select text in the Source area with the keyboard and the mouse (it remains read-only), and its selection is kept when the focus moves to the Translation area. *F9* translates the source selection — or the whole source when nothing is selected — by assembling the **fragments** known to the active dictionaries (§ 12.8), text-only and personal dictionaries first: the whole text first, otherwise, word

after word, the longest known fragment starting there (case-insensitive); a word that no fragment covers is copied as is and counted as unknown; engine tags, numbers and punctuation are copied. With a selection, the result is inserted at the cursor of the Translation area (or replaces its selection); without a selection, it replaces the translation, after confirmation if the translation is not empty ("Do not ask again during this session" checkbox). The initial capital follows the destination: at the start of the translation, it follows the source; in the middle of a sentence, a fragment taken from the start of the source loses its capital ("Une" + "Steel Sword" → "Une épée en acier"). The line is set to Pending, the operation is undone like typing, and the message line gives the summary: "2 fragment(s) translated, 1 unknown word(s) (open)". *Ctrl+F9* translates in two parts: the cursor in the Source area splits the text (a cursor inside a word splits after that word), each part is translated and the two results are joined by a space; without a cursor, Ctrl+F9 is the same as F9. In the grid, F9 does nothing: line menu ▸ Apply the dictionaries to the selection… (§ 11.3) remains the grid's entry point.

8.6 Find and replace

Tools ▸ *Find and replace…* (*Ctrl+H*) opens a separate window, only one, tied to the active document, pre-filled with the selection of the cell being edited or, failing that, the filter text. Two fields, *Find* and *Replace*, each with the ▾ button that drops down the last ten patterns of the session. *In* chooses the target — Translation, Source, EDID or FormID, Signature — but only the translation can be replaced in a plugin: on another target, the Replace buttons are grayed out and say so, while searching remains possible. *Scope*: the whole document, the filtered view or the selection (with the number of lines). Options: *Match case*, *Whole word*; modes: *Normal*, *Extended* (`\n`, `\r`, `\t`, `\\` interpreted in the pattern and the replacement) and *Regex* (Rust syntax, groups `$1`…`$9` and `${nom}` in the replacement: "(\w+) of (\w+)" → "$2 de $1").

The preview — "37 occurrences in 29 lines — first: '…' → '…'" — is recomputed a quarter of a second after you type; an invalid expression is shown in red, with the buttons grayed out. *Next* (F3) and *Previous* (Shift+F3), from the window or from the grid, select the next or previous occurrence in the grid, which scrolls; the edit window follows it if it is open. *Replace* replaces in the current line then moves to the next one; *Replace all* processes the whole scope in a single undoable operation ("Undo: Replacement (29 lines)"). Ignored lines are never modified; a modified Validated line is set to Pending, for review, and the status bar says so: "29 lines replaced, 12 set Pending".

Apply a rules file… reads a JSON file — `{ "format": 1, "rules": [ { "find", "replace", "mode": normal | extended | regex, "case", "whole" } ] }` — and applies each rule in order over the scope, in a single undoable operation; the report, per rule, is shown below the preview. *Save as rule…* adds the current pattern to a file of your choice, created if needed; the last folder is remembered for the session. The window remembers its geometry (`[Window] ReplaceX/Y/W/H`).

Special characters. *Options* ▸ *Show special characters* — a checkbox also found in the window — makes visible, in the grid and in the edit window, the line feed "¶", the carriage return "↵", the tab "→" and the non-breaking space "°", in a dimmed color. The text is not modified and input ignores these glyphs: the saved file does contain the real characters. The setting is remembered (`[Editing] ShowSpecialChars`).

8.7 Quality checks

Tools ▸ *Quality checks…* opens a separate window, only one, tied to the active document; *F12* opens it and starts the check right away, and a second F12 runs it again. Nine checks, each with its own checkbox, whose state is remembered (`[Quality] Checks`): *Aliases* (engine tags of the source missing from the translation, unknown or in a different order), *Numbers* (numbers of the source missing or unknown), *Field maximum length* (the character or byte limit of the field table is exceeded, § 15.1), *Forbidden line break* (line break in a single-line field), *Different final punctuation* (the source ends with ".", "!", "?", "…" or ":" and the translation with something else, or the reverse), *Capital after a period* (lowercase letter after ".", "!" or "?" — unchecked by default, as proper nouns and abbreviations trigger it too often), *Double or edge spaces*, *Characters outside Windows-1252* (characters that games of the Skyrim LE / Fallout 3 generation cannot write: the list of offending characters is given) and *Different translations of one source* (the same source translated in several ways: the variants with their number of lines). Ignored lines, lines never displayed in game and lines without a translation are not checked.

Scope: the whole document, the filtered view or the selection. *Check* (F12) works in the background with a progress bar — about one second for 50,000 lines and all checks — and the interface remains usable. The result, "37 anomaly(ies) on 29 line(s)", heads a table with the columns *Line*, *EDID / source*, *Check*, *Detail*, sorted by clicking a header (an arrow shows the direction). A double-click or Enter opens the edit window on the line; Ctrl+click and Shift+click select several lines; *Fix aliases* and *Fix numbers* apply the repair of the Aliases and numbers tab (§ 8.5) to the selected lines, in one undoable operation. An "Inconsistency" line has the *Unify on the most frequent* button: all lines of the group, Ignored excluded, receive the majority variant, in a single undoable operation (a Validated line goes back to Pending). The window remembers its geometry (`[Window] QualityX/Y/W/H`, 700 × 520 by default).

In the grid, each flagged line shows *[!]* in red in the status cell, with a tooltip listing its anomalies; the *[!] Anomalies* button of the filter bar (§ 4.3) shows only those lines. The indicator follows your corrections: validating an input, repairing, replacing or unifying recomputes the checks of the line (and the window's table) without rerunning the whole set — only the inconsistency check, which compares lines with each other, waits for the next F12. The anomalies belong to the checked document: switching tabs clears them, F12 recomputes them.

8.8 Dialogues

A dialogue is scattered across the plugin: a quest (QUST) owns topics (DIAL), each topic contains replies (INFO), and each reply carries one or more responses (NAM1 text, emotion, author's note), the player's paraphrase (RNAM), a speaker, conditions and links to the neighboring replies; scenes (SCEN) chain topics by phases, with actors designated by alias. The grid shows these lines in file order. *Tools* ▸ *Dialogues…* (*F10*) opens a separate window, only one, tied to the active document, which rebuilds the tree and lets you follow a conversation reply by reply. It lives with the document: switching tabs makes it follow, and a document without dialogue opens it on "No dialogue in this document.".

The context pass. On first request — Dialogues window, Dialogue tab of the edit window, NPC filter — the program re-reads the plugin in the background and derives the dialogue map from it: quests and aliases, topics, replies, scenes, NPCs (name, gender, voice type), voice types, factions, keywords and global

values named by the conditions. About one second for 50,000 lines, with progress shown in the window. Forms from master files (an NPC from Skyrim.esm, another mod's quest) are named when the master is found — in the plugin's folder, then in the Data folder of the game detected or set in Settings ▸ Paths: an index of its names is built once, in the background, and kept in `cache/names/` in the configuration folder (key: file size and date, source language); without the master, the form is shown as "000A2C8E (Skyrim.esm)".

The tree. First level: the plugin's quests (EDID — name), then "No quest" for topics that have none. Under a quest: the scenes and their phases ("Phase 1 — Ralof"), each phase leading to the replies of the action played, then the topics by category and subtype ("Topic — Custom", "Misc — Hello", "Combat — Attack"…), each topic with its EDID and its text ("MQ101RalofTopic01 'What's going on?'"); a reply shows its speaker, the start of its first response and the emotion; a reply with several responses can be expanded. Nodes carry the status dot of their lines (gray when the responses do not all have the same status) and the "translated / total" counters. The labels of categories, subtypes and emotions come from the `assets/data/dialogue/<family>.json` file (skyrim, fallout4, fallout3, oblivion, starfield, morrowind).

The reply. The right-hand pane shows the chosen reply: FormID, topic, rank in the topic; the **speaker** with gender, voice type and how it was resolved — *ANAM* when the reply names its NPC, *scene* when it is a scene actor (the quest alias, resolved through its unique actor or its forced reference), *condition* when a condition says so (GetIsID → the NPC, GetIsAliasRef → the alias, GetIsVoiceType → "voice MaleNord", GetInFaction, GetIsRace), several possible speakers being listed with "or"; otherwise "undetermined speaker" (generic greetings); the quest, the topic and its category, the listener; **the player's question** (RNAM, or the topic text) with its translation; the **responses** in order, each with its number, its emotion and intensity ("Anger 60"), its source, its translation or "(to translate)", and the author's NAM2 note; the **conditions** in plain language; the **voice** (§ 8.9). Clicking a response selects its line in the grid; double-clicking opens it in the edit window.

Conditions in plain language. Each condition of the reply is decoded through the function table of its game family (`assets/data/conditions/<family>.json`: index, name, parameter types) and stated in plain English for the sixty or so common functions — "if the speaker is Ralof", "if quest MQ101 is at stage ≥ 20", "if the player is a woman", "if quest MQ101 is completed", "if the speaker has the keyword …" — or otherwise in the generic form "Function(parameters) op value"; a function missing from the table is shown as "Function #4096". Conditions are joined by "and", and the plugin's OR flag by "or"; a condition on the listener or on a reference says so first ("on the listener: …"); a global value is named. Hovering over a condition shows its raw form ("GetIsID([01000A01 Ralof], 0) == 1"). The Skyrim and Fallout 4 tables are complete; Oblivion, Fallout 3 / New Vegas and Starfield have the names of the common functions, and the others are shown by their number.

Following a conversation. *Next* goes to the next response or reply of the topic (file order is the game's priority order), then to the first reply of the first linked topic (TCLT); in a scene, to the next action of the phase, then to the next phase; *Previous* follows the PNAM link when there is one, otherwise goes to the previous reply. *Ctrl+Alt+Left* / *Ctrl+Alt+Right* do the same in the Dialogues window and in the Dialogue tab; *Enter*, a double-click or *Open in the editor* open the edit window on the line, and it then follows the

navigation. Selecting a reply selects the line of its first response in the grid; with *Follow the grid* (checked by default, `[Dialogues] FollowGrid`), selecting a line in the grid selects the corresponding node.

Filters and NPCs. The *Quests* list and the *NPCs* list (the document's resolved speakers, with their gender and number of responses, the most talkative first), the search in the sources and translations of the replies (a quarter of a second after you type) and the *Unvoiced* checkbox (§ 8.9) reduce the tree to the matching nodes; the window's status bar counts "60 replies · 12 translated · 4 unvoiced · 3 NPCs · filter: …". *Filter the grid on this NPC*, next to the NPC list, and *Filter on this NPC* in the line menu (on a reply whose speaker is resolved) set a grid filter that combines with the others, shown as the "NPC: Lydia ✕" chip in the filter bar; Reset removes it. The window remembers its geometry (`[Window] DialogX/Y/W/H`, 900 × 580 by default) and the position of its splitter (`[Dialogues] Split`); Esc closes it.

Flat, in the grid. The Comment column (§ 8.1, *Options ▸ Columns ▸ Comment*) summarizes this context line by line — speaker and gender, quest, topic, emotion, response rank —, and *In: Comment* in the filter bar (§ 4.3) searches it: "(f)" for replies spoken by a woman, "Anger" for lines played in anger, the name of an NPC.

8.9 Voices

You translate a voiced reply with in mind the subtitle that scrolls during the English voice; a reply without a voice is a gap the mod author will want to know about. On first request — Dialogues window, Dialogue tab, Comment column, "voice ✗" filter — the program builds the mod's **voice map** without extracting or reading the audio: it lists the loose files under `<Data>/Sound/Voice/<plugin name>/` (Data is the plugin's folder, which covers a mod opened from its Mod Organizer or Vortex folder) and all the archives in the plugin's folder whose name starts with the plugin's name (`MonMod.bsa`, `MonMod - Voices.bsa`, `MonMod - Voices_en.ba2`…), of which only the directory is read. The game's archives are not consulted. The map is kept with the document; *Refresh*, at the bottom of the Dialogues window, rebuilds it after files are added.

Matching. A voice file is recognized by the suffix "_<FormID as eight hexadecimal digits>_<response number>.<ext>" (fuz, xwm, wav, wem, mp3, ogg, lip depending on the game): the low twenty-four bits of the FormID are compared with those of the reply — the load index written in the name depends on the author's load order, so "0501A3F2" matches "0101A3F2" —, and the number with that of the response. The parent folder gives the voice type (Skyrim, Fallout 4) or race / sex (Oblivion, "Imperial/M"). When a response has several voice types, all are listed. A `.lip` file alone is not a voice ("lips only"); a `.fuz` file contains both voice and lips. Starfield names its `.wem` files by hash: nothing is matched, and the window says "unresolved (Starfield: N .wem files)". These conventions are in the `voice` section of `assets/data/dialogue/<family>.json` (root folder, extensions, naming mode, folder depth).

Display. In the Dialogues window, each response has its own line: "✓ 1 · MaleNord · MQ101_MQ101RalofTopic01_01000301_1.fuz (loose)", "✓ 2 · malenord · … (MonMod - Voices.bsa)", or in red "✗ 2 · no voice file" and "✗ 1 · lips only (…lip)"; the tooltip gives the full path, or the archive and the internal path; below the conditions, the *Voice* section repeats them, with *Open the folder* for a loose file. The Dialogue tab of the edit window gives the same line, abbreviated; the Comment column (§ 8.1) ends the reply with "voice ✓" or "voice ✗", which you can filter with *In: Comment*. A reply

counts as **unvoiced** as soon as one of its responses has no voice file: the window's status bar counts them, so do the tree counters, and the *Unvoiced* checkbox keeps only them. *Copy the list of unvoiced replies* puts one line per reply on the clipboard — “FormID ⇥ topic ⇥ speaker ⇥ text” — for you to send to the mod author.

No source. A mod without a Sound/Voice folder or archive (or with an empty folder) has no known voice: nothing is marked ✗, the counters stay at zero and the window says “No voice file for this mod (neither Sound/Voice/MonMod.esp nor an archive).”. Audio playback and the 3D viewer are planned for step 3.

8.10 Translating online

A machine translation service takes ten seconds to produce the first draft of a three-thousand-word book and leaves you the review, where your value is real. The program connects to **DeepL** (free or pro), **Azure Translator**, **OpenAI** and any server that speaks the same API (LM Studio, Ollama, vLLM… on your machine) and **Google Cloud Translation**. Three principles: nothing is sent without an action on your part; what is sent is announced (lines, characters, requests); what comes back is marked *Pending* with its provenance and is undone with Ctrl+Z. The dictionaries (§ 7.3) remain the first resort; the service handles what they leave.

Configuring. *Settings* ▸ *Online* (§ 16.4): choose a service, paste its key, set its endpoint and options, then *Test…* — DeepL answers with your usage for the month, the others with a control sentence; an error is reported with the service's code and message. A service is usable when its key is present (or when its local server responds); the *Default service* is the one Ctrl+G calls. Each service has its own terms and pricing: DeepL offers 500,000 characters per month with a free key, the others are billed by usage. The key is stored in the system keyring — Windows Credential Manager, macOS Keychain, Secret Service on Linux — and, without a keyring, in `secrets.json` in the configuration folder, readable by you only; never in Config.ini, never in the log.

The line. *Ctrl+G*, the line menu ▸ *Translate online* or the *Online* button of the edit window send the source of the current line — or of the selected lines, as one batch — to the default service, in the document's Source → Target pair. The answer replaces the translation, with the status *Pending*, provenance “online (DeepL)” (status tooltip, Record tab, Comment column), as one undoable operation (“Undo: Online translation (1 line)”); a *Validated* or *Custom* line is replaced only after confirmation. Without a usable service, Ctrl+G opens the browser on Google Translate, as in the Translate a FOMOD window, and the status bar points you to the Settings; in that window, Ctrl+G also uses the service when there is one.

The batch. *Ctrl+Shift+G* or *Tools* ▸ *Translate online…* opens the dialog: the *Scope* (current line, selection, filtered view), the *Only the lines to translate* box (checked: the To translate lines and the empty Pending lines; unchecked: the whole scope except Ignored and never-shown lines), the service, then the announcement of what will be sent: lines and distinct sources (a repeated source is sent only once and its answer is placed on all its lines), characters, number of requests according to the safeguards, number of aliases, numbers and line breaks protected, the session counter, and the lines the cache will serve without sending anything. *Translate* starts sending in a background thread: the progress bar advances request by request, *Stop* interrupts after the current request and keeps what has been received, and the log lists each line received or refused (service error, lost token, empty answer or answer identical to the source, line

too long). The received lines come in as one undoable operation (“Undo: Online translation (184 lines)”). The mod dictionary is touched only on save, as for any line.

What is protected. Before sending, in each source, the game's aliases and tags (`<Alias=…>`, `<Global=…>`, book tags…) and the placeholders of text files (`{0}`, `%s`) become `<a_A>`, `<a_B>`…; numbers become `<n_A>`…; line breaks become `<br_A>`…. The services that allow it receive these tokens as tags not to be translated (DeepL, Azure, Google); OpenAI receives the instruction. On return, each token gets its value back; a missing, duplicated or altered token makes the answer refused: the line is not modified and the log says so. The order of the tokens may have changed — the target language sometimes requires it —, which A-07 reports as a warning, not as an error. Each of the three protections can be unchecked in the Settings (numbers, for example, for a language where the service reformats them usefully).

Safeguards. Characters per request (5,000), lines per batch (50) and requests per minute (20) split and pace the sending; a line longer than the limit is sent alone, never truncated, and is refused if it still exceeds it. A counter of the characters sent since launch is shown in the dialog; beyond the threshold (200,000, 0 to never ask), each new batch asks for confirmation. Each answer is kept in `cache/online/<service>_<SRC>-<DST>.ximdb` in the configuration folder: a source already translated by the same service in the same pair is not sent again, even after a restart; *Clear the cache* erases it. The proxy from the Settings, or failing that the HTTPS_PROXY environment variable, is honored; the timeout is adjustable; requests and answers are not logged.

8.11 Reviewing a translation with someone else

Review by a second person is where a mod translation gains its quality. Rather than passing the same file back and forth — whoever saves last overwrites the other —, the reviewer has a column of their own, *Review*, and the translator accepts the proposals line by line. *Options* ▸ *Columns* ▸ *Review* (`[Columns] ShowReview`, unchecked at first) shows it between Translation and Provenance; the edit window then shows a Review field below the translation.



Figure 8.11 — The Review column: in blue the proposals that differ from the translation, in grey an accepted proposal; the Review sub-menu of the line menu (French interface).

- **Typing a review**: the Review cell is edited like the Translation cell, and so is the field of the edit window. Changing the review touches neither the translation, nor the status, nor the provenance; it is an undoable “Review” operation.
- **Accepting**: *Ctrl+Shift+R* or the line menu ▸ *Review* ▸ *Translation = Review*, on the line or the selection: a non-empty review that differs replaces the translation, the line becomes Validated (Pending if it equals the source) with the provenance “review” (§ 9.1). The review stays, in grey, until *Clear the review*. A single undoable operation for the whole selection; the status bar gives the number of lines validated.
- **Starting from the translation**: *Review = Translation* copies the translation into the review, so the reviewer corrects instead of retyping everything; *Clear the review* empties it.
- **The lines to look at**: the *≠ Review* button of the filter bar, present while the column is shown, keeps only the lines whose review differs from the translation; it combines with the other filters and Reset releases it.

Loading the reviewer's copy. The reviewer works on their copy and saves it as XML or JSON (Save as…). *File* ▸ *Load into the review…* opens any file the application reads as a translation — XML, pivot JSON, SST, EET, DSD, .ximdb, or STRINGS of the target language (matched by string number) — and puts its translations into the Review column, without touching the translations. *Matching*: *Same record, field and source* (default; the FormID and EDID too when both sides have them) or *Same source (text only)*. *Fill only the lines whose review differs from the translation* (checked) leaves empty the lines the reviewer did not change; *Replace an existing review* (unchecked) protects a review typed in the meantime. Before loading, the preview counts the lines of the file, the matched ones, the reviews different from the translation, the lines without a match (source changed or missing) and the reviews already present. Loading is an undoable operation; the column shows if it was hidden.

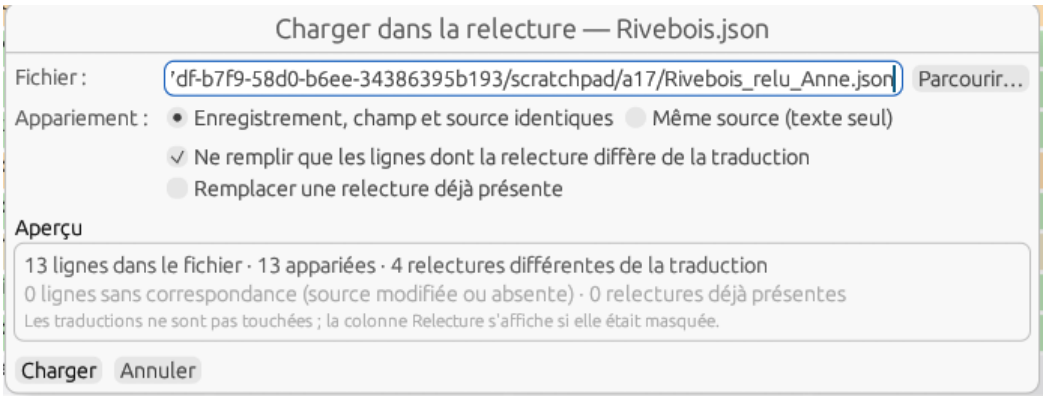


Figure 8.12 — File ▸ Load into the review…: the reviewed copy, the matching and the preview (French interface).

Where the review goes. It is saved to the mod's dictionary (format 4), the pivot JSON and the native XML; Save as… in a format that does not carry it (SST, EET, xTranslator XML, DSD, CSV, STRINGS) warns: “The review of 37 lines is not saved in this format.” Game exports, creating and applying dictionaries ignore it: only the translation counts.

9. Statuses and their colors

Each line has a status, shown by a background color and a glyph:

Status	Glyph	Meaning
To translate	•	Line not yet translated.
Validated	✓✓	Translation entered / validated.
Pending	…	Translation incomplete or in progress.
By similarity	≈	Filled automatically from a dictionary.
Modified proposal	•▬·	Automatic proposal edited by hand.
Custom	★	Marked as custom.
Ignored	∅	Line excluded from translation.

A special case: the **lines never shown in game** — the plugin header (author and description, `TES4:CNAM` / `TES4:SNAM`), the names that only Bethesda's editor uses (explosions, projectiles, hazards, and depending on the game classes, factions, trees or lights), and, in Fallout 4, Fallout 76 and Starfield, the names of the **object templates** of weapons and armor (the predefined combinations of modifications, extra `FULL` lines of a `WEAP` or an `ARMO`) marked “editor only” — only the template of a unique legendary, which names the object in game, stays translatable and carries the comment “Object template”. They have the Ignored status, are shown on a red background (the Ignored color) with the source text in place of the translation, cannot be edited and receive nothing from the dictionaries: translating them would be wasted work. They are hidden by default; the *Never shown in game* box of the filter bar shows them (§ 10). The list is per-game data (“Never shown in game” mode in the field editor, § 15.1), deliberately cautious: if a mod author tells you about other useless lines, add them to it.

The colors are fully customizable via *Options* ▸ *Status colors…*. The picker offers an *Index color* tab (AutoCAD ACI palette) and a *True colors* tab (RGB). *Reset colors* restores the default palette.

9.1 The provenance of translations

The status tells where a line stands; the **provenance** tells where its translation came from. Every operation that writes a translation sets it on the lines it changes, and only on those: *typed* (typed in the grid or the edit window, pasted, Translation = source); *dictionary*, with the file, its role (game, mod, personal, text, ext., backup) and the match level (record and text identical, same text, text only, old data, same record with text changed); *heuristic* with its score (Use button of the Vocabulary tab, instant translation F9 on the whole source); *online* with the service; *propagation* (copied from a duplicate, § 8.4); *replacement* (Find and replace, alias and number repairs, quality-check unification, spell checker); *target file* (§ 7.7); *review* (Translation = Review, § 8.11); *imported* with the format, for a translation read with the file opened (SST, EET, another tool's XML). A line without translation has no provenance.

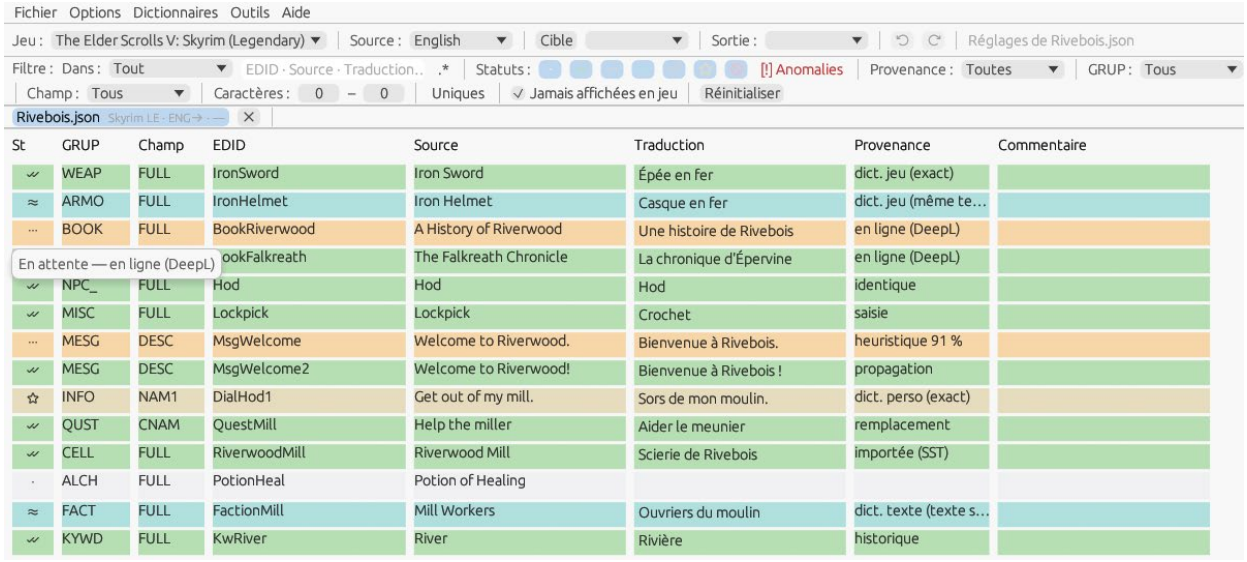


Figure 9.1 — The Provenance column (Options ▸ Columns) and, on the status cell, the "Status — provenance" tooltip (French interface).

- **Changing the status does not change the provenance**: a line received from DeepL then validated (F5) stays "online"; the tooltip says "Validated — online (DeepL)". Editing by hand a translation that came from elsewhere turns it into "typed"; clearing the translation clears the provenance; Ctrl+Z and Ctrl+Y restore it with the translation and the status.
- **Where to read it**: the tooltip of the status cell ("Status — full provenance", for example "Validated — Skyrim_ENG-FRA.ximdb (dict. game), record and text identical"); the Record tab of the edit window (§ 8.5); the Provenance column, as a short label ("dict. mod (same text)", "heuristic 91 %").
- **Identical**: a non-empty translation equal to the source (outer spaces aside) shows as "identical" in the column and falls under "Same as source" in the filter, whatever its saved provenance — which stays in the tooltip. This is the case to review: a proper noun left as is, or an oversight.
- **Saved with the work**: the provenance is written to the mod's dictionary (.ximdb format 4, an eighth element per line; formats 1 to 3 read as before and a file moves to format 4 at its first save), to the pivot JSON (save and autosave) and to the XIMOD native XML. A line found again at reopening through the mod's dictionary gets back the provenance it had ("online (DeepL)"), instead of "mod dictionary". The formats of other tools (SST, EET, xTranslator XML, CSV, STRINGS) and shared dictionaries carry none.
- **No new color**: the provenance is read in the text; the seven status colors are unchanged.

10. Filters and counters

The filter bar restricts the lines shown; the criteria combine:

- *Filter* (text): shows only the lines containing the text entered, in the target chosen with *In* (§ 4.3) — all, source, translation, EDID or FormID, signature, or the generated comment (§ 8.1).
- *Statuses*: limits the display to certain statuses.

- *≠ Review*: while the Review column is shown, only the lines whose review differs from the translation (§ 8.11).
- *Provenance*: All, Typed, Game dictionary, Mod dictionary, Personal dictionary, Other dictionary (text-only, external, backups), Heuristic, Online, Propagation, Replacement, Review, History, Imported (target file included), Same as source, Untranslated (§ 9.1). It combines with the statuses: “Online” + Validated shows the lines of a service that you have already validated.
- *GRUP*: limits to one record type.
- *Field*: limits to one subfield.
- *Characters*: filters by text length.
- *Unique*: a single line per distinct source (with propagation, § 8.4).
- *Never shown in game*: also shows the lines the game never displays (§ 9), in red and not editable; unchecked at first, the box is remembered (`[Editing] ShowUnused`).
- *Record …* / *Source …*: chips placed by the line menu (§ 8.2) — all the lines of one FormID, or of one source text; a click on the chip removes it.
- *Reset*: clears all the filters.

The status bar counters (shown / unique / to translate / translated) reflect the result of the filtering live.

11. Saving and exporting

11.1 Save as…

Menu *File* ▸ *Save as…* saves the work in a translation database format: `.sst`, `.eet`, `.dsd`, `.json`, `.csv`, XML or STRINGS. The format is chosen by the extension selected in the save dialog. Useful to keep a reusable memory or to resume later. The formats that have a comment column — EET, EET XML, XIMOD XML — receive the author's comment as it was read; with *Options* ▸ *Also export the generated comments (EET, XML)* (`[Export] GeneratedComments`, unchecked at first), the generated text of the Comment column (§ 8.1) is added after it, line by line — so a colleague's file receives no text they did not write, unless you ask for it; if the dialogue map is not built yet, it is built at that moment. The other formats and the mod dictionary never carry a generated comment.

11.2 Export translation…

Menu *File* ▸ *Export translation…* produces the game-ready result, according to the document's *Output* (§ 6.1). A dialog comes before the choice of the destination file. For a text document (§ 7.7), the dialog adds the *Untranslated lines: keep the source* box and the proposed file is `<mod>_<target language>.txt`.

Exported lines

Only the lines with the *Validated* status are exported; the others keep the source text. The *Also include* group lets you also take the *By similarity*, *Pending*, *Modified proposal* or *Custom* lines; the boxes are unchecked each time the dialog opens and a warning reminds you that these lines have not been reviewed. The count updates with each box (“1,022 lines will be exported (812 validated + 210 by similarity)”) and the end-of-export message repeats it. The *To translate* and *Ignored* lines are never

exported. For the outputs that rewrite the text (STRINGS, hybrid, full plugin), a line that is not exported keeps its source text; for the override patch and the DSD, it is absent, and a record appears only if it has at least one exported field.

> **Note —** This behavior differs from previous versions, which exported every translated line: the lines filled by the dictionaries ("by similarity" status) are no longer exported without your review, unless you check the corresponding box.

Override patch of a localized plugin

For a localized plugin (Skyrim SE, Fallout 4, Starfield), the *Override patch* mode is available: the patch is written **delocalized** — translated records only, plain texts (the translation for the exported fields, the original text resolved from the STRINGS for the other text fields), "localized" flag removed, original plugin declared as the last master. The dialog shows the number of records and of unresolved ids (fields written empty if you continue) and offers to flag the patch as *ESL* (light) when the game and the patch allow it.

STRINGS file names

The translated STRINGS files follow the game's convention: `MonMod_French.strings` for Skyrim and Fallout 4, `MonMod_fr.strings` for Starfield (two-letter codes). For Starfield, place them in `Data/Strings` and enable loose files in `StarfieldCustom.ini` (`[Archive] bInvalidateOlderFiles=1`, `sResourceDataDirsFinal=`).

11.3 Apply the dictionaries…

Options ▸ *Apply the dictionaries…* — or, from the line menu, *Apply the dictionaries to the selection…* — applies the document's active stack (§ 7.3, § 12.8) or a file of your choice (.ximdb, or a raw EET, SST, DSD, JSON, XML database read into memory; the chosen file is kept for the session) to a *Scope*: the whole document, the filtered view or the selection, with their line counts; the lines never shown in game and the Ignored lines are always out of scope. *Overwrite* says what may be replaced: Nothing (empty lines only, the behavior at opening), Everything but Validated and Custom, Everything but Pending, Pending only, Everything; a line the dictionaries do not know keeps its translation and its status, even with Everything. *Reset first* empties the translations of the scope (Validated and Custom excluded, except with Overwrite = Everything) before applying, in the same operation; this is how you regenerate the by-similarity lines from the current dictionaries (the former "Re-translate auto lines" entry no longer exists).

Matching: **Strict** — identical record, field, FormID, EDID and text give Validated (the stored status for the mod dictionary, Custom for the personal dictionary), then, in the same dictionary, the same text of the same field gives By similarity: the rule applied at opening; **Relax** — same record and field, same text → By similarity; **Text only** — same text, whatever the field, signed lines or not → By similarity; **FormID only** — same record, field, FormID and EDID but a different source text: the translation of the old text, with the Pending status since the line has changed — this is the mode for mod updates; **Approximate** — the word heuristic of the Vocabulary tab (§ 8.5): each line of the scope receives the best close source from all the dictionaries (on an equal score, the first in the stack), with the By similarity status and the provenance "approximate (75 %)", above the threshold set in Settings ▸ Advanced and recalled in the label. The **preview** is recomputed a quarter of a second after each change (in the background with a progress indicator beyond 10,000 lines; in Approximate mode, after the word indexes are built, which is shown):

“1,027 lines would receive a translation (812 strict, 215 relax), 0 overwritten”, and in Approximate mode the breakdown “1 at 90 % and above, 0 from 80 to 89 %, 0 below 80 %”; Apply is grayed out at zero. Applying is a single undoable operation (“Undo: dictionaries applied (1,027 lines)”), a toast gives the summary, each filled line carries its provenance (§ 7.3) and the anomaly indicators (§ 8.7) are recalculated. From the line menu, the dialog opens on the selection with Overwrite = Everything but Validated and Custom and Strict; the other settings are those used last time, except the scope.

12. The Dictionaries menu

A **dictionary** is a file in the `.ximdb` format that associates a source text (original language) with its translation. XIMOD Translator uses it to automatically translate the entries of a mod that match the game's text or an existing translation. This menu gathers five families of operations: **converting** dictionaries between XIMOD and external tools; **building** a dictionary from a game, a mod, the STRINGS, a pair of plugins or a pair of locres files; **exporting** a game dictionary to STRINGS; **viewing** a dictionary read-only; and **managing** the list of installed games.

> **Note —** The source and target languages are **never** chosen in these dialogs: they always come from the *Source* and *Target* lists of the main window. They set the direction of the dictionaries created and the file suffix (e.g. `ENG-FRA`).

12.1 Before you start

The settings of this menu come from two places: the main window (languages and game) and *Options* ▸ *Settings* (tool and database folders). Some entries stay grayed out as long as the folder of the corresponding tool is not set:

- **Source / Target languages** — in the main window: they determine the direction and the naming of the dictionaries.
- *Game* — at the top of the main window: XIMOD cannot guess which game a mod belongs to (masters such as `Skyrim.esm` are shared). The chosen game loads its dictionary and sets the destination subfolder. **As long as the four lists are not set, the whole Dictionaries menu is grayed out.**
- **ESP-ESM Translator folder**: required for EET ↔ ximdb conversions.
- **xTranslator folder**: required for SST ↔ ximdb conversions.

Where the dictionaries are stored

Each game has its own subfolder under `assets\bases\jeux`. It contains the base game dictionary, named `<Game>_ENG-<LANG>.ximdb`, and a separate dictionary for each DLC, under its name. One file is created for each language pair (English being the source). For example:

- `assets\bases\jeux\SkyrimSE\SkyrimSE_ENG-FRA.ximdb` (base game)
- `assets\bases\jeux\SkyrimSE\Dawnguard_ENG-FRA.ximdb` (DLC)
- `assets\bases\jeux\SkyrimSE\SkyrimSE_ENG-DEU.ximdb` (another language pair)

Mod dictionaries, for their part, are stored in `assets\bases\mods\<Game>\`. The game subfolders are: `Fallout3`, `Fallout4`, `Fallout76`, `FalloutNV`, `Morrowind`, `Oblivion`, `OblivionRD`, `Skyrim`, `SkyrimSE`, `Starfield`. `OblivionRD` stands for Oblivion Remastered, kept separate from the original Oblivion. The list of DLCs per game is in `assets\data\GamesDLC.json`, which you can edit to add DLCs or Creation Club content.

12.2 Menu overview

Menu entry	Purpose
Convert an EET / SST to ximdb	Import a dictionary from an external tool into the ximod format (.ximdb).
Convert a ximdb to EET / SST	Export a ximod dictionary to the format of the external tool.
Build a dictionary from a game…	Compile into a ximdb the raw databases found in a game folder.
Build a dictionary from a mod…	Build a ximdb from mod files (EET/SST/DSD/JSON/XML).
Build a dictionary from the game STRINGS…	Pair source and target STRINGS into a game ximdb.
Build a dictionary from a pair of plugins…	Pair two plugins (same content, two languages).
Build a dictionary from a pair of locres files…	Oblivion Remastered: pair two locres exports (CSV/JSON).
Export a game dictionary to STRINGS…	Rewrite translated STRINGS from a ximdb.
View a dictionary…	Browse a .ximdb (source → translation, with a filter). The only command of the menu available without a selected game.
Create a text-only dictionary from a file of pairs…	A “source<TAB>translation” file → a text-only .ximdb (§ 12.9).
Manage dictionaries…	The dictionary tree of the active document: roles, boxes, priority order, added files and folders (§ 12.8).
My personal dictionary… / Edit a dictionary…	The dictionary editor, on the personal dictionary of the pair or on a chosen .ximdb (§ 12.10).
Mod update…	The document compared with its mod dictionary: added, removed, changed; old translations taken back (§ 12.11).
Save the mod dictionary	Write the working memory of the active document now (§ 7.6), without waiting for a save or an export.
Detect installed games… / Add a game…	Search for or register the Bethesda games.

12.3 Converting between XIMOD and external tools

Four submenus, each with *Single file…* (one file) and *Batch…* (several files). **EET** conversions require the ESP-ESM Translator folder; **SST** conversions require the xTranslator folder.

Conversions concern only **mod dictionaries**: game dictionaries are built directly from the installed games (§ 12.4) or shipped with the application for the games that require external handling (Morrowind, Oblivion…). Each tool stores its mod dictionaries in one folder per game: `Bases Persos\<game>` for ESP-ESM Translator, `UserDictionaries\<game>` for xTranslator, `assets\bases\mods\<game>` for XIMOD Translator. It is this *<game>* folder that decides the destination.

Direction **EET/SST → ximdb**: the file picker opens in `Bases Persos` (EET) or `UserDictionaries` (SST). With *Single file…*, choose a file; with *Batch…*, choose either the root folder (all the *<game>* subfolders are processed) or a *<game>* folder. Each dictionary is written to `assets\bases\mods\<game>`, the mod name and the languages being derived from the file name. The *Convert to ximdb* dialog shows an *Example* of the result and asks for the game only if a file is not in a recognized *<game>* folder.

Direction **ximdb → EET/SST**: the file picker opens in `assets\bases\mods`; the converted file is written directly to the tool's `Bases Persos\<game>` or `UserDictionaries\<game>` folder, without further questions, under the name of the source dictionary. A `.ximdb` taken from outside a *<game>* folder is refused with a message.

Mapping of the *<game>* folders between ESP-ESM Translator and XIMOD Translator (xTranslator uses the same names as XIMOD):

ESP-ESM Translator	XIMOD Translator / xTranslator
Fallout 3	Fallout3
Fallout 4	Fallout4
Fallout 76	Fallout76
Fallout New Vegas	FalloutNV
Morrowind	Morrowind
Oblivion	Oblivion
Skyrim	Skyrim
Skyrim SE	SkyrimSE
Starfield	Starfield

> **Note —** If a conversion submenu is grayed out, the folder of the tool (ESP-ESM Translator or xTranslator) is not set in the Settings.

12.4 Building a dictionary

> **Note —** Because the subfolder follows the game of the active document (or the default game when no document is open), and not the actual content of the files, a warning *"Is this really a dictionary for <Game>?"* is shown when saving: *Validate* saves, *Cancel* abandons. This applies to the five "Build…" entries.

> **Note —** Timestamped copies — when a mod dictionary overwrites an existing file of the same name, the old one is first moved to the `backups/` subfolder under the name `<name>_<YYYYMMDD-HHMMSS>.ximdb`; the ten most recent copies are kept (number adjustable in *Settings ▸ Advanced*, 0 to disable). The `.ximdb` dictionaries are written in format 2, which can carry the status of each line; format 1 files are still read.

From a game…

Select a game folder; XIMOD compiles the raw databases it contains into a dictionary stored under `assets\bases\jeux\<Game>\`.

From a mod…

Select one or more mod files (`EET, SST, DSD, JSON, XML`). The dictionary is stored under `assets\bases\mods\<Game>\`, named after the first file.

From the game STRINGS…

Choose the source STRINGS (source language), then the target STRINGS (target language) — multiple selection is possible. The entries are paired by their identifier; only the pairs that are actually translated are kept.

From a pair of plugins…

Builds the dictionary from two identical plugins in two languages — typically for old games (Morrowind, Oblivion, Fallout 3/NV) whose text is in the ESM. The window offers two fields (*Source plugin* / *Target plugin*, with *Browse…*), a *Destination* choice (Game or Mod), and a reminder of the languages. The fields are remembered until the window is closed, and the window **stays open after each save** (you chain the master, then each DLC); it closes only with *Close*.

The file is named after the source plugin (e.g. `Anchorage.esm` → `Anchorage_ENG-FRA.ximdb`). Both plugins must have the **same file name**; a trailing language code (e.g. `MonMod_FR.esp` vs `MonMod.esp`) is ignored. If the names really differ, or if no entry matches, an alert is shown and nothing is saved.

From a pair of locres files (CSV/JSON) — Oblivion Remastered

For Oblivion Remastered, the game text is in `Game.locres` files (Unreal) packed in the `.pak` files; automatic generation and the plugin pair do not apply. First extract the text of each language with an external tool, in one of the two accepted formats:

- JSON from **FModel** (the simplest): export the `Game.locres` of one language; shape `{ namespace: { key: text } }`.
- CSV from **UEExtractor**: extract each language by targeting its folder, e.g. `--path=…/Localization/Game/fr/` (otherwise the languages merge and English wins).

Extract the source and the target in the **same format** so that the keys match (codes present: `de, en, es, fr, it, ja, pl, pt, zh-Hans`). In XIMOD, choose *OblivionRD* as the *Game*, set *Source*/*Target*, then *Build a dictionary from a pair of locres files…*: give the source file, then the target file, the destination, and *Build dictionary*. XIMOD pairs by key and saves only the translated entries; the file `OblivionRD_ENG-FRA.ximdb` is stored under `assets\bases\jeux\OblivionRD\`.

12.5 Exporting a game dictionary to STRINGS

Export a game dictionary to STRINGS… does the reverse of a generation: choose a `.ximdb`, then the source STRINGS folder, then an output folder. XIMOD rewrites translated STRINGS files from the dictionary.

12.6 Managing games

Detect installed games… runs a search on all the recognized platforms (§ 2.4: Steam, GOG and GOG Galaxy, Epic, Heroic, Lutris, Xbox, folder scan; when several find the same game, the most reliable source is kept), with a progress bar, shows a summary (game · source · path) and saves the paths in `Config.ini`. *Add a game…* registers a game folder manually, validated by the presence of its master file, then locked (kept by later detections).

12.7 Worked examples

Building the dictionary of an old game (Oblivion)

1. Set *Source* and *Target* (e.g. ENG → FRA), then choose *Oblivion* in the *Game* selector.
2. Menu *Dictionaries* ▸ *Build a dictionary from a pair of plugins…*, *Destination* set to *Game*.
3. Give the English `Oblivion.esm` (source), then the French `Oblivion.esm` (target).
4. *Build dictionary*: `Oblivion_ENG-FRA.ximdb` appears in `assets\bases\jeux\Oblivion\`. Repeat for each language.

Converting an EET dictionary to ximdb

Single file… converts a single `.eet`; *Batch…* takes a folder and gathers all the `.eet` files it contains (subfolders included).

1. Check the ESP-ESM Translator folder in the Settings, and set *Source* / *Target*.
2. Menu *Dictionaries* ▸ *Convert an EET to ximdb*, then *Single file…* or *Batch…*.
3. Choose the `.eet` file in `Bases Persos\<game>` (Single file), or the `Bases Persos` / `Bases Persos\<game>` folder (Batch).
4. In *Convert to ximdb*, check the *Example* (the destination *<game>* folder is derived), then *Convert*.

12.8 Managing dictionaries: the tree

Dictionaries ▸ *Manage dictionaries…* opens, for the active document, the tree of its dictionaries: six roles — Mod, Personal, Text only, Game, External, Backups — each with its files, a box per role and per file, and an order that is the priority order (§ 7.3). An unchecked role disables its files without losing their boxes. *Move up* and *Move down* move the selected role or file; *Add a file…* accepts a .ximdb — or a raw EET, SST, DSD, JSON, XML database, read into memory — from any folder, *Add a folder…* takes all the .ximdb files of the target language from it, without copying anything; these additions go into External and are the only ones that *Remove* removes (the files of the XIMOD folders can only be unchecked). A .ximdb that declares a language other than the target is reported and ignored. *View…* (or a double-click) opens the file in the viewer (chapter 13).

The detail line describes the selected file: role, number of lines, format, date, and the number of document lines that owe their translation to it. *Save* stores the tree for this game and this language pair

([Dictionaries] Dicts_<Game>_<SRC>-<DST>): a file that has appeared since then takes its default place, checked (except in Backups), and a file that has disappeared is removed silently. *Apply to the document* stores the tree, then replays the stack on the empty lines, as one undoable operation. *Reset the order* returns to the initial order, everything checked except the backups, additions forgotten (to be saved). Closing with changes asks: Save, Discard the changes, Cancel. The box of the personal dictionary is the same as the one in the context menu of the tab (§ 7.3). The window geometry is remembered ([Window] DictsX/Y/W/H).

12.9 Text-only databases

A **text-only** database is a dictionary whose lines have no record, no field, no FormID and no EDID: text → text pairs ("of Burning" → "de feu"). The .ximdb format allows this since its version 3; a file whose lines are all of this type takes the Text only role in the tree (§ 12.8), whatever its folder; a mixed file keeps the role of its folder and its unsigned pairs are still used. In each dictionary, after the exact match and the match by text of the same field, a text-only pair recognizes a line whose source text is identical (apart from leading and trailing spaces), whatever the field, and gives it By similarity; the provenance says "text only". They are also the material for instant translation (B-14, upcoming). *Dictionaries ▸ Create a text-only dictionary from a file of pairs…* reads a UTF-8 text file, one "source<TAB>translation" pair per line (empty lines and lines starting with # are ignored), and writes it to mods/<game>/<file>_<SRC>-<DST>.ximdb after the game is confirmed, like the other builders. The personal dictionary (B-04, upcoming) will be a text-only database.

12.10 My personal dictionary and the dictionary editor

The personal dictionary of the language pair — dict/personal_<SRC>-<DST>.ximdb in the configuration folder — is your own terminology, which follows you from mod to mod ("Dwemer", "Dovahkiin", the ranks of a faction). It is fed in three ways. *Ctrl+M* (line menu ▸ Add to my personal dictionary, in the grid on the selection or in the edit window on the line) sends it the source → translation pair of each translated line, as text only by default (§ 12.9): the terminology then applies to any field; the *Ctrl+M keeps the field (REC:FLD)* option of Settings ▸ Advanced ([Personal] KeepField) stores the pair with its signature. A pair already known with the same translation is left as it is; with another translation, a dialog asks Replace, Keep or Cancel, with the "Remember this choice for the session" box; beyond twenty lines, a confirmation gives the number. The file is written at once, with a timestamped copy, and the toast says "3 pairs added (1 replaced)". Second way, the *Fill my personal dictionary at each save* option ([Personal] AutoFill, unchecked at first): at each successful Save as… and export, the Validated and Custom lines whose pair is not yet known go into it as text only, and the status bar says so; Pending, By similarity and Ignored lines never go into it. Third way, the editor.

Dictionaries ▸ My personal dictionary… opens the dictionary editor on the personal dictionary of the current pair (created empty if it does not exist); *Dictionaries ▸ Edit a dictionary…* and the *Edit…* button of the tree (§ 12.8) open it on any .ximdb. An independent window, only one at a time: a filter (substring in the source or the translation, applied a quarter of a second after typing), a Source / Translation / Field / Status table that can be sorted by clicking the header, and a Source, Field and Translation editing area. A click on a pair loads it into the area; *Add* creates a pair, *Replace* modifies the selected pair (source and

field included), *Delete* removes it; Enter means Replace when a pair is selected, Add otherwise; Del in the table deletes. An empty field makes a text-only pair, “REC:FLD” a signed pair. Everything stays in memory until *Save* (timestamped copy, dictionaries of the open documents reloaded); closing with changes asks. *New…* creates an empty text-only dictionary for the pair in `mods/<game>` or in the chosen folder. A game dictionary shipped with the software opens read-only; *Save as…* makes a copy of it in your folders. On a mod dictionary that has old data (§ 7.6), the *Old data (N)* box switches the table to it, with the disappearance date in the Status column; *Delete* removes one entry, *Put back among the current rows* returns it to the current lines. The geometry is remembered (`[Window] DictEditX/Y/W/H`).

12.11 Mod update

Dictionaries ▸ *Mod update…* compares the active document with its mod dictionary (current lines and old data): at the top, the date the dictionary was written, its number of lines and that of the open file; then the summary — **added** (unknown to the dictionary, record included), **removed** (in the dictionary, no longer in the file: the current lines will move to the old data at the next write, the old ones are already there with their date), **changed** (same record, field, FormID and EDID, different source text) and unchanged — and three tabs that list them; the list of changed lines gives the old source, the new one and the old translation, and a double-click opens the line in the edit window. *Take the N old translations* gives the changed lines whose translation is empty or by similarity their old translation, with the Pending status, as one undoable operation; the lines already Validated, Pending or Custom are kept and counted. *Forget the removed…* empties the old data section after confirmation (the previous file goes to `backups/`). Without a mod dictionary, the window offers to write it. When you open a file some of whose lines have changed since the dictionary, the status bar says so without applying anything. The geometry is remembered (`[Window] UpdateX/Y/W/H`).

13. The dictionary viewer

Menu *Dictionaries* ▸ *View a dictionary…*. It opens in a separate window whose size and position are remembered.

- Open a `.ximdb` with *Open…* or by dragging it onto the window; *Close* unloads the file.
- A *Filter* field narrows the display; a counter shows the number of lines.
- Two equal columns, *Source* and *Translation*, with long texts wrapped.
- A **striping** (alternating rows, as in a spreadsheet) whose color is set with *Options* ▸ *Dictionary viewer stripe color…*.

14. The spell checker

14.1 Installing a dictionary

The spell checker relies on Hunspell and on a pair of `.aff` / `.dic` files for the *Target* language (e.g. `fr_FR.aff` / `fr_FR.dic`), to be placed in the folder shown by the window.

14.2 Opening and using the spell checker

Menu *Options* ▸ *Spell checker…*. For each error, you can apply a suggestion, *Ignore* the word for the session, or add it to the personal dictionary (+ *Dictionary*). *Recheck* runs the analysis again; *Reload* reloads the dictionary.

14.3 The personal dictionary

Added words (proper names, game terms…) are kept and are no longer flagged as errors in later checks.

15. The explorers (Tools menu)

The Tools menu also opens Find and replace (§ 8.6), Quality checks (§ 8.7) and Dialogues (§ 8.8). The three explorers open in separate windows, positioned near the main window with a slight offset; their size and position are remembered, and all three can be open at the same time.

- *ESP/ESM/ESL Explorer*: browses the structure of a plugin (GRUP, records, subrecords).
- *BSA/BA2 Explorer*: lists the contents of a Bethesda archive. You can show only the *Translatable only* entries, *Extract…* a file, *Open in translator*, or *Open associated plugin* (resolves its STRINGS from the archive).
- *SWF Explorer*: explores Flash interface resources (SWF).

> **Note —** An archive can also be opened directly by dragging it onto the main window (`.bsa` / `.ba2`).

15.1 Tools ▸ Data ▸ Translatable fields per game…

What is extracted as a translatable line is described, game by game, in a shipped data file (`assets/data/fields/<Game>.json`) and no longer in the program. The editor shows the effective list for the chosen game — one line per `RECORD:FIELD` signature (or per EDID or text pattern), with the mode (*Yes* = extracted and editable, *Locked* = extracted but with the Ignored status by default, *Never shown in game* = extracted with the Ignored status, in red, hidden by default and not editable — plugin header, editor-only names (§ 9), *No* = never extracted), the non-empty constraint, the maximum length, whether line breaks are allowed (Yes, No, or "—" as long as this is not known: the edit window then lets Enter through), an optional context rule ("EDID starts with s" for game settings, for example) and a comment. Grayed lines come from the parent game (Skyrim SE inherits from Skyrim, Fallout 76 from Fallout 4, Oblivion Remastered from Oblivion); editing a line makes it an entry specific to the game.

You edit the selected line in the bottom panel; validation is continuous (signature in the AAAA:BBBB format, uniqueness, well-formed rule) and an invalid file is not saved. *Save* writes only your differences from the shipped version, to `data/fields/<Game>.json` in the configuration folder — the shipped file is never modified — and the changes apply the next time a file is loaded. *Restore shipped version* deletes your file. An unreadable user file is reported at startup and the shipped version is used.

Engine tags. The `tags` key of the file, at the head of the field list, lists the tag names compared by the Aliases and numbers tab (§ 8.5); a name ending in * covers any tag that begins that way (`Alias*` covers `<Alias=…>` and `<Alias.Name=…>`). An empty list compares all <…> tags; the list of a child game replaces the parent's. It is edited in the file; the editor does not show it yet.

Shipped values. The tables specify line breaks for the fields whose behavior is certain: forbidden in names (FULL, Morrowind FNAM, short names, button and objective labels, activation verbs), allowed in descriptions (DESC, book text, effect descriptions); the others remain "—". Length limits are set only when they are those of the engine and not of an editor: Morrowind names, at 31 characters (fixed game buffer). The limits of the Creation Kit editor (149 characters for a dialogue response, 80 for a topic) do not constrain the game: they appear in the rule comment, for information. Adjust these values in the editor if your experience differs.

15.2 Tools ▸ Data ▸ Text formats…

Text file formats (§ 7.7) are described in `data/text_formats.json` in the configuration folder, initialized from `assets/data/text_formats.json`. Each format has an identifier, a name, **name patterns** ("Interface/Translations/*_*.txt", "translate_*.txt", "*.ini"; a pattern containing "/" is compared with the end of the path, the others with the file name), an **encoding** (auto, utf-8, utf-16le, utf-16be, windows-1252; auto reads the byte order mark, then tries UTF-8, then Windows-1252 — a mark, when present, always wins), **line endings** (keep, crlf, lf), the **line expression** (Rust syntax) with the named groups `key` and `value`, an **escaping** (none, json, c: "\"" and the line breaks of the value), the **Skip empty values** option and **Language in the name** (suffix: `<base>_<language>`; none). Shipped: MCM / translate, INI, flat JSON (one ""key": "value"" pair per line), CSV key;value.

The editor lists the shipped formats (padlock, read-only — *Duplicate* makes an editable copy) and yours; *New* creates a blank format, *Delete* removes the selected format. *Try on a file…* applies the expression to a file and reports the number of lines recognized out of the total, with the first three pairs. *Save* rejects an invalid expression (with the error message) or one without the key and value groups, and writes only your formats. When a file is written, the value is replaced in place in the recognized line — everything around it ("key = ", quotes, comma) is kept — which makes a write template unnecessary. The window remembers its geometry (`[Window] FormatsWin*`).

15.3 Tools ▸ Translate a FOMOD…

FOMOD installers — the ones XIMOD Architect builds — carry texts that the player reads in Mod Organizer 2 or Vortex: the name and description of the mod (`fomod/info.xml`), the module name, the names of the steps, groups and options, and the option descriptions (`fomod/ModuleConfig.xml`). This window translates them; it is the same as the one in XIMOD Architect, built on the library shared by both applications, and it works on the same file: a translation started in one reopens in the other.

Opening. *Open a mod folder…* accepts the mod folder or its `fomod` subfolder; *Open a mod archive…* accepts a zip or a 7z, of which only the `fomod` folder is extracted (into the configuration folder) — the "full mod" package is then not possible. Dropping a folder or an archive onto the main window also opens this window, as does `--fomod <folder>` on the command line. The source and target languages are chosen before loading and are fixed afterwards; the translator name is remembered (`[General] Translator`). The work is saved in `fomod/translations/<mod>.<language>.ximod-translation` (Ctrl+S): reopening the same FOMOD in the same language finds it again, and *Update from folder* merges a new version of the mod with the translation (new, changed, moved, removed texts).

The table. One line per text: number, status (○ to translate, ✓ translated, ◐ from memory, ~ to review, ✕ obsolete, 🔒 locked), context (step › group › option), source, translation editable in place for names, and ⚠ when a check has something to report. The *All / Untranslated / To review / With problems / Locked* filters, *Unique texts* (one line per distinct text; what you type propagates to identical ones), the field type and the search narrow the list; ↑ ↓ PgUp PgDn Home End move the line when you are not typing, *Next untranslated* (Ctrl+Enter) jumps to the next one to do. The bottom panel shows the chosen line: source and translation side by side (descriptions are translated there, over several lines), *Clear*, *Copy source*, *Translate online* (Ctrl+G: the configured online service puts the translation in the target, to be reviewed; without a service, Google Translate in the browser, to be copied back — § 8.10), *Do not translate* (lock), the memory suggestion, the counter, the detected problems (missing tag, empty text, line break in a name, abnormal length, inconsistent duplicate…) and a note.

Helpers. *Fill from memory* proposes the translations already saved for the same texts (the memory learns at each save, per language pair); *Propagate to identical* copies the line's translation to all identical texts; *Export CSV… / Import CSV…* for a spreadsheet or an external translator; *Glossary* enforces the translation of a term, or keeping it as is, and flags deviations; *Copy the source list* and *Paste translations…* do the round trip with the AI of your choice as a numbered list ("translate this list into French, keeping the numbers").

Exporting. *Export the translated files* writes the translated XML files without losing anything from the originals (comments, encoding, layout, unknown elements) according to the chosen mode: into a `fomod_<lang>` folder next to it, in place, or as a zip / 7z package ready for Nexus (with a readme, the full mod if the whole folder is available, and a configurable name). *Keep the original order* keeps the explicit order of the steps. A blocking problem prevents the export and selects the *With problems* filter. Closing the window with unsaved changes shows a Save / Discard / Cancel row in the window; the ✕ button in the title bar is the only way to close it (not Esc, so that no input is lost).

15.4 Tools ▸ Package a translation…

Once the translation is exported (§ 11.2), this window builds the folder and the `.zip` archive that Mod Organizer 2 and Vortex install, according to the game family (§ 18): fill in the game and the language (prefilled from the toolbar), the mod name as published, the version, your translator name, the website and the output folder, then add the files. The preview shows the package tree and the notes to read before publishing; a file with no known place for this game is flagged and left out.

File	Place in the package
Plugin (.esp / .esm / .esl)	At the root (Data folder). An override patch must stay below the original mod in the load order.
STRINGS (Skyrim, Skyrim SE, Fallout 4, Fallout 76, Starfield)	Strings/; exact file names, language included. Starfield: loose files must be enabled in StarfieldCustom.ini.
DSD (.json, Skyrim SE)	SKSE/Plugins/DynamicStringDistributor/<target plugin>/; requires SKSE and Dynamic String Distributor.

File	Place in the package
BSA / BA2	At the root, named after the plugin that loads it.
Morrowind	At the root of Data Files; the translated plugin replaces the original.
Oblivion Remastered (PAK / UCAS / UTOC)	OblivionRemastered/Content/Paks/~mods/; the three files go together. A plugin goes under OblivionRemastered/Content/Dev/ObvData/Data/.

The package is named “<mod> - <LANGUAGE> - <version>”; *Create the folder and the archive* writes both to the output folder (an existing folder with the same name is recreated) and, if the box is checked, a `fomod/info.xml` with the name, the translator, the version, the website and the description (by default “<language> translation of <mod>”). *Open the folder* shows the result.

15.5 Tools ▸ Translate the interface…

This independent window lets you translate or complete a language of the interface (§ 2.6) without editing files by hand or knowing the Fluent format, then send it to the project. At the top: the **flag** opens the countries window, and the country offers its languages in *Language to translate*; only a language with an ISO 639-3 code in `Languages.json` can be translated, and a language without a folder is created when saving. *Displayed language* chooses the language shown alongside, any existing language — changing it keeps what you typed. Then come the country endonym (“Bundesrepublik Deutschland”, “Schweiz” depending on the flag), the language endonym (“Deutsch”), the author and the *Font*: *Browse…* picks it inside `assets/fonts` (a font stored elsewhere is refused: copy it into that folder first), *Google Fonts…* opens the site to download one; the font is used at once for the endonym fields, the Translation column and the preview.

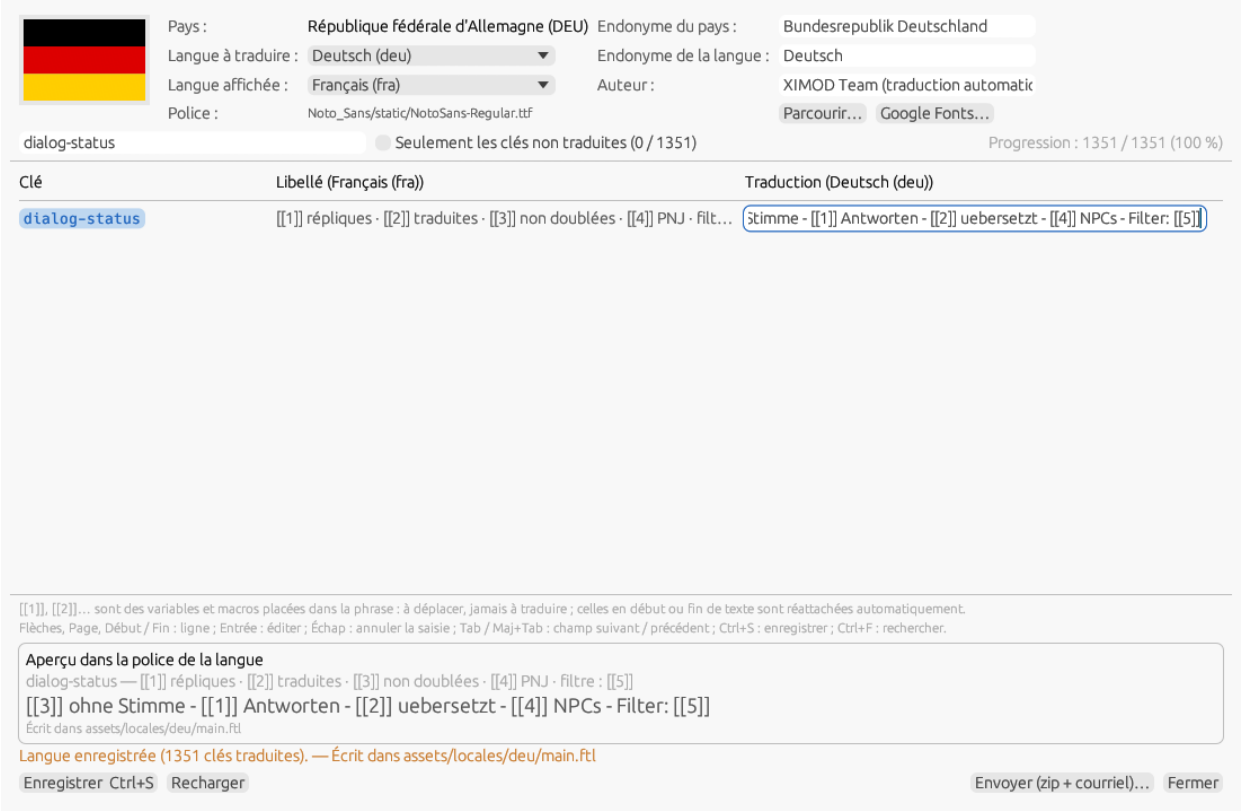


Figure 15.5 — Translate the interface: German, with French alongside; the dialog-status key has just been edited and saved.

The table has three columns: *Key*, *Label* in the displayed language (the tooltip gives the full text) and *Translation*, the only editable one. The search field filters on the key, the label and the translation; *Only untranslated keys* (with their number) keeps the empty translations only — a row you fill in stays shown until the filter changes again; the progress ("1,028 / 1,240 (83 %)") is on the right. Fluent variables (`{ $count }`) and macros (`$MODNAME$`) are never typed: at the start or end of the text they are hidden and re-attached on save with their separators; in the middle of the text they become markers `[[1]]`, `[[2]]`… that you move within your sentence. A lost, unknown or duplicated marker turns the row red (the tooltip says which): the key will then take the label on save. Untranslatable keys (`app-title` and the `# @untranslatable` list of the reference file) and keys made of variables only are hidden; plural or multi-line messages are shown as they are, in italics, and cannot be edited here. The preview, at the bottom, shows the whole current row in the font of the language.

Keyboard. Arrows, Page Up / Down, Home / End change the row; Enter edits the row's translation, Esc cancels the current edit; Tab / Shift+Tab go to the next / previous row (to the untranslated rows when the filter is checked); Ctrl+S saves; Ctrl+F goes to the search.

Save writes `assets/locales/<code>/main.ftl` in the order of the reference file (English), with its blank lines and comments — those of the language when its file follows the reference —, headed by `# @language`, `# @font`, `# @langname`, `# @author`; a key left empty takes the label, so the file is always complete. The language endonym and the font go into `Languages.json`, the country endonym into `Countries.json`, only if you changed them; the country endonym is also recorded in `countryEndonyms.tsv` next to the file. When `assets/` cannot be written (system installation), the save

goes to `<configuration folder>/locales/<code>/main.ftl`, which the application reads before the one in `assets/`; the “Written to …” line of the preview always says where. The language appears at once in Settings ▸ Interface language and, if it is the current one, the interface reloads. *Reload* reads the files again, discarding unsaved changes after confirmation; closing the window with changes also asks for confirmation.

Send (zip + e-mail)… builds `ximod-translator_<code>.zip` in the configuration folder — `locales/<code>/main.ftl` and `countryEndonyms.tsv`, never the ISO data files —, opens that folder and a pre-filled e-mail (project recipient, subject “XIMOD Translator — traduction <language>”, author and path of the archive): attach the archive and send; nothing leaves without you. The window remembers its geometry (`[Window] TrWin*`).

16. Settings

The *Settings* window (menu *Options* ▸ *Settings…*) has four tabs: General, Recent files, Online and Advanced.

16.1 General tab

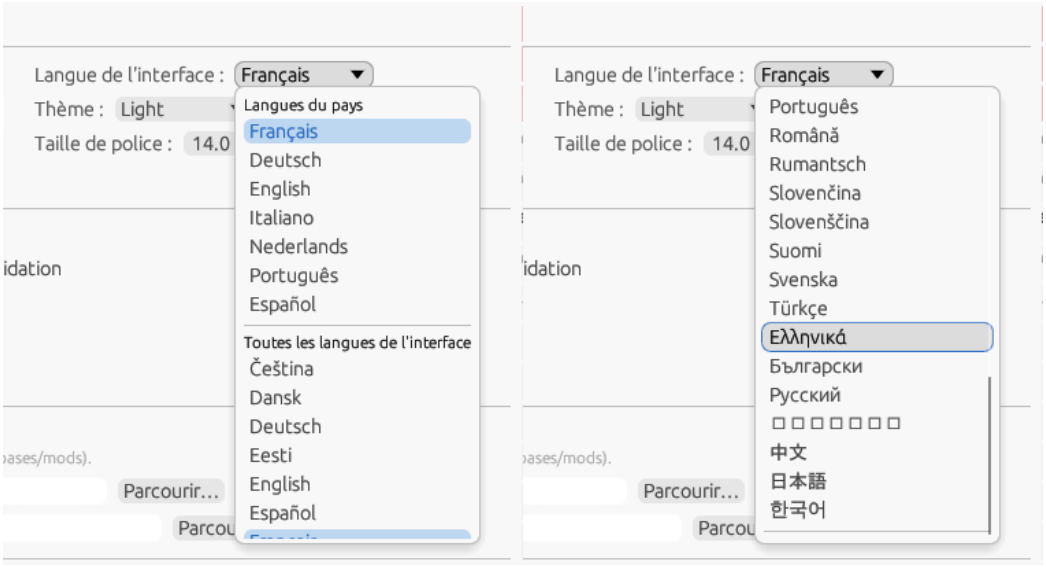


Figure 16.1 — The interface language: the languages of the chosen country (France), then all languages, each in its font (screenshots of this manual are in French).

- *Theme*: light, dark or system.
- *Interface language*: the list first offers the *Languages of the country* chosen with the flag that have an interface, then *All interface languages*, each under its own name and in its own font, in alphabetical order (§ 2.6); applied immediately. The code is stored in `[General] Locale`.
- *Font size*: previewed live while you adjust it.
- *Max recent files*: number of entries kept in the Recent menu.
- *Window width / Window height*: dimensions of the main window.

- **Translation database folders** — *Game bases* and *Mod translations*; leave empty for the default locations (`assets/bases/jeux` and `assets/bases/mods`).
- **External tools** — root folders of *ESP-ESM Translator* and *xTranslator*. An empty path disables the corresponding conversions in the Dictionaries menu.

As in XIMOD Architect, Save and Cancel stay visible at the bottom of the window whatever the height of the tab; the control that has keyboard focus is outlined; the theme and the font size are previewed while you adjust them and reverted on cancel (Esc); the language applies only on save. *Transient notifications* turns the messages at the top right on or off (§ 4.5); *Propose the propagation to the lines of the same source after a validation* opens the propagation dialog after F5 or Ctrl+↓ (§ 8.4).

16.2 Recent files tab

Lists the recently opened files; you can clear the list.

Removing an entry or clearing everything is saved immediately, without waiting for Save; *Clear* asks for a second click (“Click again to clear”) before acting.

16.3 Advanced tab

- **Backups** — *Automatically save the work in progress* (enabled), interval in minutes (5, from 1 to 60), *Timestamped copies kept per mod dictionary* (10, from 0 to 100) and *Rescue folder* (empty = `autosave/` in the configuration folder), with a button to open it, and *Write the mod dictionary at each save and export* (checked; § 7.6), *Fill my personal dictionary at each save* (unchecked) and *Ctrl+M keeps the field* (unchecked; § 12.10) and *Keep the old data of the mod dictionaries* (0 = forever; § 7.6). See § 7.5 and § 12.4.
- **Word heuristic** — *Word heuristic threshold*: proportion of differing words allowed between a line and a close source (20%, from 5 to 50; § 8.5 and § 11.3).
- **Configuration location** — *Per user (recommended)* or *Portable* (§ 2.5); the current location is shown with an *Open folder* button. Changing it copies the configuration, the personal dictionaries, the data and the backups to the destination and takes effect at the next startup; switching to portable is refused if the executable's folder is not writable.
- **Undo** — number of undoable operations kept per file (200, from 10 to 1,000).

16.4 Online tab

The online translation services (§ 8.10). On the left, the four services with the state of their key; the *Default service* list sets the one that Ctrl+G calls. On the right, the selected service: *API key* (hidden, *Show* / *Hide*), *Endpoint* (DeepL: api-free.deepl.com for a free key, api.deepl.com otherwise; Azure: the endpoint and the *Region*; OpenAI: the endpoint — api.openai.com or a local server — and the *Model*, the key being optional for a local server), *Formality* (DeepL), and *Test…*. The *Guards* group holds the quotas, the session threshold and the three protections; the *Network* group, the proxy and the timeout; *Clear the cache* erases the stored responses. Keys are saved with *Save*, in the system keyring or, failing that, in `secrets.json` — the line under the key says which.

16.5 Keyboard navigation

Dialog boxes can be driven from the keyboard: Tab to move around, arrows for drop-down lists, Enter to confirm, Esc to close.

Shortcut	Function
Ctrl+Z / Ctrl+Y	Undo / Redo (§ 6.6).
F5 / F6 / F7 / F8	Validated / Pending / Custom / Ignored status for the selection (§ 8.2).
Ctrl+↓ / Ctrl+↑	Validate, then next / previous line (§ 8.2).
Shift+click / Ctrl+click	Extend the selection / add or remove a line.
Right-click	Line menu (§ 8.2); Ctrl+C / Ctrl+V / Ctrl+F8 when not typing: copy, paste, translation = source.
↑ ↓ / PgUp PgDn / Home End	Move the selection by one line, by one page, to the start or to the end of the view (when not typing).
Ctrl+O / Ctrl+S / Ctrl+Shift+E / Ctrl+W / Ctrl+Q	Open / Save as / Export / Close document / Exit (§ 4.1).
Ctrl+, / F1	Settings / User manual.
Ctrl+F, Esc	Focus on the Filter field with its text selected; Esc in the field clears it (§ 4.3).
Ctrl+H, F3 / Shift+F3	Find and replace; next / previous occurrence (§ 8.6).
Ctrl+M	Add the pair of the selection, or of the edit window's line, to my personal dictionary (§ 12.10).
F10	Dialogues: opens the dialogues window of the active document (§ 8.8).
Ctrl+Alt+Left / Ctrl+Alt+Right	Previous / next line of the conversation, in the Dialogues window and the Dialogue tab (§ 8.8).
F12	Quality checks: opens the window and runs the checks; in the window, Enter opens the selected line (§ 8.7).
Enter, double-click	Open the edit window on the line, or bring it to the front (§ 8.5).
Ctrl+Enter	Edit window: validate, then next line (§ 8.5).
Alt+Right / Alt+Left	Edit window: next / previous line without changing the status (§ 8.5).
Ctrl+F6	Apply to every line with the same source… — propagation with dialog, grid and edit window (§ 8.4).
F9 / Ctrl+F9	Edit window: instant translation of the source selection or of the whole source / in two steps, at the source cursor (§ 8.5).
Esc	Edit window: close while keeping the input; close a dialog.
Ctrl+S, Ctrl+Enter, Ctrl+G	Translate a FOMOD window: save, next untranslated, translate the line online (§ 15.3).

17. The configuration file

All preferences are saved in `Config.ini`, in the configuration folder (§ 2.5), and restored at the next launch. The file is managed automatically and written atomically (never truncated); it is human-readable and repairs itself if a value is missing. Its main sections:

Section	Contents
`[General]`	Interface language, country, theme, font size, number of recent files, splash duration, Toasts (transient notifications), Translator (name signed in FOMOD translations); ConfigMode (portable / user), informational.
`[Translation]`	Last choices in the Source, Target, Output and Game (DefaultGame) lists, remembered for information only: at launch, the lists start empty; PersonalDictDefault (initial state of the personal dictionary box).
`[Editing]`	UndoDepth: undoable operations kept per file; ShowUnused; EditTab: last tab of the edit window; ShowSpecialChars: special characters visible; ProposePropagation: propagation dialog after validation (§ 8.4).
`[Columns]`	Optional grid columns (§ 8.1): ShowComment (Comment column), ShowReview (Review column, § 8.11), ShowProvenance (Provenance column, § 9.1); 0 by default.
`[Export]`	GeneratedComments: generated comments are written to EET and XML files (§ 11.1); UntranslatedCopy: untranslated lines of a text file taken from the source (§ 7.7).
`[Dialogues]`	FollowGrid: the Dialogues window follows the grid selection; Split: position of its splitter (§ 8.8).
`[Quality]`	Checks: selected quality checks, separated by commas (§ 8.7).
`[Fuzzy]`	Threshold: word heuristic threshold, as a percentage of differing words (§ 8.5).
`[Personal]`	AutoFill: personal dictionary filled at each save; KeepField: Ctrl+M keeps the field (§ 12.10).
`[Dictionaries]`	Dicts_<Game>_<SRC>-<DST>: the dictionary tree of each game and language pair — roles and files in order, “!” for an unchecked box, full path for an added file, “+” before an added folder (§ 12.8).
`[Backup]`	AutosaveEnabled, AutosaveMinutes, AutosaveDir, Copies; ModDictAuto: mod dictionary written at each save and export; OldDataMonths: old data kept (months, 0 = forever) (§ 7.6).
`[Online]`	Default service, endpoints and service options (DeepLEndpoint, DeepLFormality, AzureRegion, AzureEndpoint, OpenAIEndpoint, OpenAIModel), quotas (MaxCharsPerRequest, MaxLinesPerBatch, MaxRequestsPerMinute, SessionWarnChars), protections (ProtectTags, ProtectNumbers, ProtectNewlines), Proxy, TimeoutSeconds (§ 8.10, § 16.4). API keys are never stored there: system keyring or secrets.json.

Section	Contents
`[Window]`	Size and position of the main window and of all secondary windows (Settings, viewer, explorers, spell checker…).
`[Paths]`	Folders of the databases (games/mods) and of the external tools (EET, xTranslator).
`[Games]`	Games detected or added manually.
Colors & recent files	Status colors, stripe color, list of recent files.

> **Note —** If the application behaves unexpectedly, closing it and then deleting `Config.ini` restores all default settings at the next launch.

18. Localization of Bethesda games

The way a game stores its texts determines how to translate it. There are four families:

Family	Games	Where the texts are
A — embedded strings	Morrowind, Oblivion, Fallout 3, New Vegas	Directly in the ESP/ESM fields (one language per file, Windows-1252 encoding).
B — Bethesda STRINGS	Skyrim LE, Skyrim SE/AE, Fallout 4	The “Localized” plugin contains identifiers; the text is in the .STRINGS/.DLSTRINGS/.ILSTRINGS triplets.
C — centralized STRINGS	Fallout 76, Starfield	ESM + a multilingual “… - Localization.ba2” archive.
D — Unreal LOC	Oblivion Remastered	LOC_* keys, Game.locres and Unreal StringTables in the PAK/UCAS/UTOC containers.

A mod from family B or C can nevertheless be **non-localized** and contain its English text in plain form in the ESP: XIMOD Translator detects the encoding automatically on opening. On the installation side, a translation is distributed according to the family: translated plugin, “loose” STRINGS triplet, localization archive, or Unreal/PAK files — making sure, in MO2 or Vortex, that the translation wins over the original.

The three sections that follow extend this table with concrete cases: where the application looks for the texts of a localized plugin, how to know the language of a plugin, and how XIMOD Translator can be used beyond translating a mod — up to giving a game a language that Bethesda never released.

18.1 Where the application finds the STRINGS

When a localized plugin is opened, XIMOD Translator first looks for the **loose files** `<plugin>_<language>.strings`, `.dlstrings` and `.ilstrings` in the `Strings` folder next to the plugin; if they are missing, it opens the **neighboring BSA or BA2 archives** whose name begins with the plugin's name (`MyMod.ba2`, `MyMod - Main.ba2`) and extracts the STRINGS from them. A mod downloaded as is,

whose author packed the texts into its archive, therefore opens without any manual steps. The same path exists in reverse: *Tools* ▸ *BSA/BA2 Explorer* (or dropping an archive onto the window) lists the contents of an archive, and *Open associated plugin* loads the neighboring plugin, resolving its texts from the archive (§ 15).

> **Note —** Loose files always win over the archive: this is what makes it possible to install a translation as "loose" STRINGS without touching the original BA2 — provided the game loads loose files (for Starfield, enable this in `StarfieldCustom.ini`).

18.2 Knowing the language of a plugin

For a **localized plugin**, the answer is in the files: each STRINGS file carries its language in its name (`_english`, `_french`…; `_en`, `_fr`… for Starfield), and the application lists them. This is how it checks that the chosen *Source* language really exists next to the plugin, or in its archive, and refuses to open the file otherwise (§ 6.3). A plugin can thus offer several source languages.

For a **non-localized plugin**, the format says nothing: a plugin header has no language field, and the plain text can be in any language — in practice, almost always English. The *Source* language is then **detected from the text** by the Lingua library, which recognizes 75 languages from their letter sequences and remains reliable on short texts: the list receives the plausible language or languages, the most likely first (§ 6.3). When in doubt — a mod made only of short names, often proper names — two clues help you decide: the mod page (its author states the language), and the grid itself, where the names of records taken from the base game (a renamed NPC, a rebalanced weapon) can be compared at a glance with the text of the game dictionary.

18.3 Adding a language that Bethesda does not offer to a game

XIMOD Translator translates mods, but nothing limits it to mods: the game's ESM and its DLCs are plugins like any other, and the output modes make it possible to give a game a language that the publisher never released. The method depends on the game family.

Games with STRINGS (Skyrim, Skyrim SE/AE, Fallout 4, Fallout 76, Starfield)

The engine does not have a closed list of languages: it reads a token in the ini file (`sLanguage=ENGLISH` in `Skyrim.ini`, and its equivalent in `Fallout4.ini` or `StarfieldCustom.ini`) and loads, for each localized plugin, the file `<plugin>_<token>.strings`. Nothing prevents you from writing `sLanguage=DUTCH` and supplying `Skyrim_dutch.strings`, `Update_dutch.strings`, `Dawnguard_dutch.strings`…: communities have produced complete translations this way in languages that were never released (Brazilian Portuguese, Turkish, Hungarian, Korean…). With XIMOD Translator, this means opening each ESM of the game with *Source* set to English and *Target* set to the new language, and exporting in *STRINGS* mode (§ 11).

- The set of STRINGS must be **complete**: the ESM, all its DLCs and every installed localized mod. The engine has no fallback to English: a plugin without STRINGS in the active language displays empty texts.

- The game's **fonts** must contain the glyphs of the language: Latin diacritics and Cyrillic are included; Greek, Vietnamese or Thai require replacement fonts, and right-to-left scripts are not supported by the engine.
- The platform may rewrite `sLanguage` at launch (Steam aligns the ini with the language chosen in its settings). This is why most community translations choose instead to **overwrite the English STRINGS** while keeping `sLanguage=ENGLISH`: untranslated mods then remain readable in English, which is the best compromise in practice. With XIMOD Translator, you simply name the exported files with the English token.

Games without STRINGS (Morrowind, Oblivion, Fallout 3, New Vegas)

The texts are in the ESM itself, and a language version is a different ESM (the French `Oblivion.esm` is not the English `Oblivion.esm`). Adding a language means translating the ESM — *Full plugin* mode — or shipping an override ESP that rewrites all the records that carry text — *Patch override* mode. Both work; the patch has the drawback of conflicting with any mod that touches the same records (the last one loaded wins and brings back English), hence the value of loading it early and of translating the mods too. Fonts are replaced through the `.fnt` files in the `Fonts` folder.

Skyrim SE with DSD

The Dynamic String Distributor (SKSE) replaces texts in memory from JSON files, without touching either the STRINGS or the plugins, and therefore without load-order conflicts: for this game alone, it is the cleanest way to add a language to the ESM as well as to mods. The *DSD (SKSE)* output mode writes these files directly.

> **Note —** These indications describe the known behavior of the engines; each added language deserves an in-game test before distribution, especially for fonts.

19. Frequently asked questions

The Dictionaries menu is grayed out, and so is Open…

No game is selected. First choose a *Game* in the toolbar (§ 7.1).

The Source list is empty or does not offer my language.

It offers only the languages that the file provides: the STRINGS present for a localized plugin, otherwise the language detected in the text (§ 6.3). For a localized plugin, add the STRINGS of the desired language next to the plugin.

Nothing is translated automatically on opening.

Check that *Game* points to the right game (to load its dictionary) and that the *Target* language is correct. After opening, the status bar says which dictionaries were found and how many lines they filled; if the mod dictionary is named but fills only a handful of lines, it is because it contains only those: it is rewritten at each save and export (§ 7.6) — save or export once the translation is well advanced, or use *Dictionaries* ▸ *Save the mod dictionary*.

Changing the game reloaded my file.

This is the expected behavior: the active document is read again with the rules of the new game, and its translations are kept by matching (§ 6.2). The other documents are not affected. If you answered *Cancel* to the confirmation, nothing changed.

A "□" box appears in place of a character.

This is an old encoding problem (Latin-1 control characters), now fixed automatically on reading. Regenerate the dictionary concerned if an old file still showed it.

The spell checker says "No dictionary for the target language".

Put a `.aff` / `.dic` pair in the folder shown by the spell checker window.

20. Glossary

Term	Definition
BSA / BA2	Bethesda archives grouping resources (textures, STRINGS, interface…).
DSD	Dynamic String Distributor: SKSE framework (Skyrim SE/AE) that injects translations at launch without modifying the mod.
EDID	Editor ID: readable name of a record in the game's editor.
EET	ESP-ESM Translator: reference tool, and its .eet database format.
FOMOD	Mod installer format (handled by XIMOD Architect).
FormID	Unique numeric identifier of a record in a plugin.
locres	Localization file of the Unreal engine (Oblivion Remastered).
Plugin	.esp / .esm / .esl mod file.
SKSE	Skyrim Script Extender: extension that loads script mods.
SST	xTranslator translation database format.
STRINGS	External text tables of recent games (.STRINGS/.DLSTRINGS/.ILSTRINGS).
ximdb	Compact dictionary format specific to XIMOD.

Appendix — Summary of formats

Type	Read	Written
Translation databases	.sst, .eet, .dsd, .json, .xml	.sst, .eet, .dsd, .json, .csv, .xml
Plugins	.esp, .esm, .esl (extraction)	.esp/.esm (full plugin, override patch)
String tables	.strings, .dlstrings, .ilstrings	.strings, .dlstrings, .ilstrings
Dictionaries	.ximdb	.ximdb

Type	Read	Written
Archives	.bsa, .ba2 (browsing/extraction)	—